Online Joint Order Assignment and Task Allocation in Robotic Mobile Fulfillment Systems under Travel Time Uncertainty

Yundong Feng [a], Yiqi Sun [b] and Zuojun Max Shen [c]
[a] Department of Industrial Engineering, Tsinghua University, Beijing, China,
[b] Department of Data and Systems Engineering, The University of Hong Kong, Hong Kong, China,
[c] Faculty of Engineering and Faculty of Business and Economics, The University of Hong Kong, Hong Kong, China.

Abstract

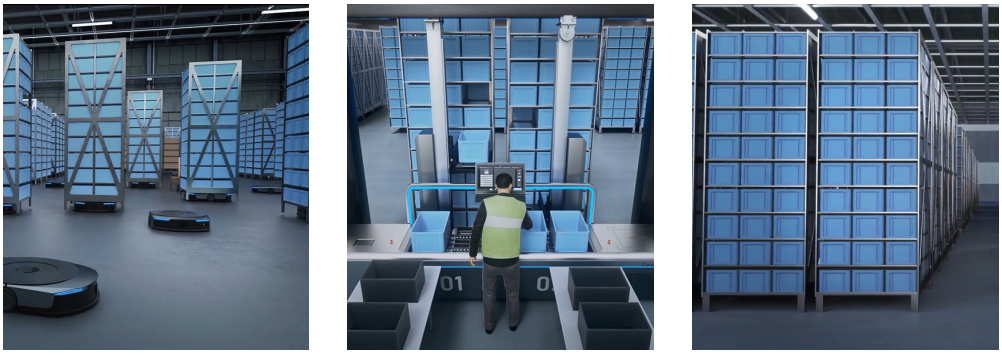
In this study, we address the online joint order assignment and task allocation problem in Robotic Mobile Fulfillment Systems (RMFS), focusing on travel time uncertainties arising from unpredictable congestion and the adaptive routing of Autonomous Mobile Robots (AMRs). Our model jointly optimizes both the sequencing of orders at workstations and the allocation of transport tasks − defined as the movement of specific pods between storage areas and workstations − to individual AMRs. To capture operational dynamics, we first formulate a stochastic programming model. To overcome computational intractability, we reformulate this model as an approximate dynamic programming (ADP) framework using Lagrange relaxation. Within this architecture, we rigorously capture the travel time uncertainty by constructing nested ambiguity sets and developing a data-driven distributionally robust optimization (DRO) model. We further design an efficient heuristic algorithm with bounded approximation guarantees. Numerical experiments based on real-world data demonstrate that our approach substantially outperforms a traditional greedy algorithm, achieving a 14.87% average relative improvement in workstation occupancy rate. These results highlight the value of explicitly modeling travel time uncertainty for real-time decision-making in high-throughput RMFS environments.

Keywords: Task Allocation, Order Assignment, Online Scheduling, Robotic Mobile Fulfillment System, Data-Driven Distributionally Robust Optimization.

1 Introduction

The Robotic Mobile Fulfillment System (RMFS) represents a significant advancement in automated storage and order picking systems, offering superior efficiency, storage capacity, and flexibility. The system's key innovation lies in its use of Autonomous Mobile Robots (AMRs), which can lift and transport "pods", i.e., shelf rack units containing inventory, between storage areas and workstations. The benefits of the RMFS are manifold. Firstly, it significantly enhances pick-up efficiency, as order picking is the most time-consuming operation in warehouses (Shah and Khanzode, 2017), and RMFS allows workers or robots

at workstations to remain stationary instead of traversing the warehouse to pick packages. In addition, because AMRs can maneuver beneath the shelves, the storage density of RMFS is greatly increased compared with traditional warehouses, leading to substantial savings in space and inventory costs. Lastly, the RMFS offers exceptional flexibility. AMRs can dynamically reposition shelves during warehouse operations, allowing for a highly adaptable inventory layout. Since Amazon first introduced the Kiva RMFS in the United States in 2012, similar systems have been developed and implemented by various companies worldwide. These advanced systems include Geek+ by JizhijiaTech in China, Butler by GreyOrange in India, and Racrew by Hitachi in Japan, etc.



(a) Transport with AMR (b) Workstation (c) Storage Area

Figure 1: Illustration of the RMFS of Geek+, China

Despite its proven efficiency and cost-effectiveness in practice, the operational management of RMFS remains highly challenging due to its inherent complexity. Even estimating the performance of a given RMFS is a nontrivial task (Lamballais et al., 2017; Yang et al., 2021). Moreover, unlike traditional Automated Guided Vehicles (AGVs), AMRs can autonomously respond to their environment and re-route without human intervention, introducing substantial uncertainty in travel times (Lamballais Tessensohn et al., 2020; da Costa Barros and Nascimento, 2021). This uncertainty is further exacerbated under high order volumes, where AMR congestion becomes difficult to predict. In addition, fundamental operational strategies in RMFS − such as task allocation, order assignment, and pod storage assignment − are computationally intensive yet require rapid responses due to the system's highly dynamic nature (Merschformann et al., 2019). The task allocation problem involves assigning transportation tasks to AMRs; the order assignment problem focuses on allocating pick-up and replenishment orders to workstations or pods; and the pod stor-

age assignment problem concerns placing pods in designated areas within the storage zone. Our research concentrates on the integrated order assignment and task allocation problem. Specifically, we jointly consider the online assignment of specified orders to workstations for processing and the allocation of the corresponding transportation tasks to AMRs.

Numerous studies have investigated operational strategies in RMFS from various perspectives. To the best of our knowledge, existing research can largely be categorized into two streams based on their formulation approaches. The first stream adopts mixed-integer programming (MIP) to formulate the deterministic RMFS problems. For example, Boysen et al. (2017) analyzes the sequencing of orders and pod visits in a setting with a single picking workstation. They formulate an MIP model to minimize the number of pod visits and propose a heuristic algorithm to iteratively generate feasible solutions. Similarly, Valle and Beasley (2021) uses MIP to model the problem of assigning orders and racks to pickers, also aiming to minimize pod usage, and proposes two heuristics for approximate solutions. In contrast, Kim et al. (2020) formulates an MIP model to assign items to pods, with the objective of maximizing the intra-pod similarity of items for operational efficiency. More recent research has shifted toward practical objectives, such as minimizing system makespan, and considers the joint optimization of task allocation and order assignment (e.g., Wang et al., 2022; Zhen et al., 2023). While MIP formulations allow for precise strategy generation, most studies remain limited to static settings due to the high computational complexity involved. Furthermore, AMR traffic congestion can cause significant delays in travel times, rendering static model solutions infeasible in real-world operations. As a result, these models typically require excessive transport capacity and oversimplified operational assumptions − conditions that are rarely realistic in practice.

The second stream of research models the RMFS as a queuing network and incorporates decision-making within a Markov Decision Process (MDP) framework (e.g., Lamballais Tessensohn et al., 2020; Lamballais et al., 2022). In these studies, random parameters are typically assumed to follow known distributions, and the system state − which includes the number of AMRs, workstations, and orders in various stages − evolves according to a transition probability matrix. This formulation allows researchers to analyze the systems long-term behavior across multiple decision epochs. While queuing network models effectively capture the aggregate stochastic dynamics of RMFS, their utility for real-time

operational decision-making is shaped by two key characteristics. First, these models typically produce high-level operational guidelines rather than the detailed, implementable decisions required for online dispatch. In other words, such models are better suited for capturing general operational dynamics rather than producing specific task or order assignments. Second, their reliance on predetermined probability distributions makes it difficult to reflect the intricate spatiotemporal dependencies characteristic of AMR congestion and autonomous navigationphenomena that are often inadequately described by conventional parametric assumptions. For instance, Lamballais et al. (2022) assume that after a pod visits a picking station, it proceeds to replenishment with a probability p, which is empirically estimated from real-world data.

To enable deployable real-time decision-making with certifiable uncertainty quantification, we address the online joint order assignment and task allocation problem in RMFS under travel time uncertainty − involving coordinated assignment of transport tasks to AMRs and orders to workstations. We formulate this integrated problem via stochastic programming and transform it into an approximate dynamic programming (ADP) framework under Lagrange relaxation. To mitigate adverse effects of congestion-induced variability, we explicitly model travel time uncertaintycapturing both task-specific and sequence-dependent variabilityusing nested ambiguity sets. This yields a data-driven distributionally robust counterpart to the ADP. We further reformulate this robust counterpart as a two-stage problem and design a polynomial-time approximate algorithm with bounded approximation guarantees. Building on this foundation, we develop an online dispatching method validated with real-world data from Geek+, demonstrating a 14.87% improvement in workstation occupancy rate over practical greedy baselines.

To the best of our knowledge, this study is the first to incorporate travel time uncertainty into online RMFS optimization using a distributionally robust optimization (DRO) framework − setting it apart from previous research that assumes fixed travel times in static scenarios (Lamballais et al., 2017) or presumes known distributions in queuing models. Beyond uncertainty modeling, our work presents a unified framework for integrated production and distribution scheduling. In contrast to conventional transportation and logistic research, we explicitly account for processing times and queue dynamics at service stations − critical factors increasingly prevalent in modern logistics systems. By leverag-

ing ambiguity sets to represent real-world transportation uncertainty, our methodology is readily extensible to emerging applications such as last-mile delivery and drone logistics. Moreover, our approach differs from prior studies through its comprehensive treatment of system component configurations. While most existing research presumes abundant AMR availability (Lamballais et al., 2017; Wang et al., 2022) − which simplifies the problem − we adopt a scheduling-theoretic perspective and systematically decompose scenarios based on the quantitative relationships among tasks, AMRs, and workstations. Our analysis focuses on the most challenging and operationally relevant case, where the number of tasks exceeds the number of AMRs, which in turn exceeds the number of workstations. This setting closely mirrors practical logistics constraints and introduces significant computational challenges that our method is specifically designed to address.

The remainder of this paper is organized as follows. Section 2 reviews the related literature on methodology. Section 3 introduces the model settings and formulates the integrated stochastic model, which is further extended into a dynamic programming model in Section 4. Section 5 presents the distributionally robust counterpart, incorporating travel time uncertainties. Section 6 provides the numerical experiments and their results. Finally, Section 7 concludes the paper and suggests directions for future research.

2 Literature Review

Beyond the operational strategies within RMFS, which have been discussed in the introduction, our research is also closely related to the integrated production and outbound distribution scheduling (IPODS) problem. This problem amalgamates the two most crucial functions in the supply chain − production and distribution − and conducts joint planning and scheduling to minimize operating costs or maximize customer satisfaction. Early IPODS problems only considered offline dispatching, where parameters such as operating time are perfectly known. For example, Potts (1980) first analyzed the offline IPODS problem with only a single machine. Woeginger (1994) further examined the IPODS problem in the context of parallel machines, providing heuristic solutions with constant worst-case guarantees, and Cheng and Kahlbacher (1993) extended the concept from individual deliveries to batch deliveries, also taking into account vehicle routing decisions and transportation

costs. Lastly, Lee and Chen (2001) explicitly considered the impacts of transport capacity and travel time, both within the flow shop and during the final product delivery process.

To address the limitations imposed by the practical setting of predetermined jobs and parameters in the offline IPODS problem, researchers have utilized continuously updated market information to revise the parameter distribution. This revised problem is referred to as the O-IPOD problem, and related studies are comprehensively reviewed in Chen and Hall (2022). In line with classic online scheduling literature, studies on the online O-IPODS problem often propose heuristic algorithms for resolution. For example, Hoogeveen and Vestjens (2000) considered a simple single-machine online scheduling problem. They established that the upper bound of the competitive ratio for an online algorithm is $\frac{1+\sqrt{5}}{2}$, and proposed a straightforward online algorithm with a competitive ratio of 2. Subsequent research considers more complex and practical scenarios, such as capacity constraints, distribution costs, batch processing modes, and limited distribution capacity in batch delivery modes (Pundoor and Chen, 2005; Tian et al., 2007; Averbakh, 2010). To our knowledge, the study by Tang et al. (2019) bears the closest relevance to our work. They addressed the integrated optimization problem of production and two-phase distribution, proposing both offline and online algorithms and calculating their respective competitive ratios. In this scenario, the transportation of products from the warehouse occurs in two stages: initially from the factory to a transit hub, followed by delivery from the transit hub to the customer's location. In contrast to prior studies, our work introduces a key structural distinction: a closed-loop process that requires bidirectional (inbound and outbound) transport operations. Specifically, AMRs must (i) retrieve pods from storage and deliver them to workstations (inbound), (ii) queue and undergo processing at workstations (production), and (iii) return processed pods to storage areas (outbound) before starting new tasks. This contrasts fundamentally with the conventional (open-loop) production $\rightarrow$ outbound sequence in IPODS. Consequently, our closed-loop system (inbound-production-outbound) inherently induces sequence-dependent travel times due to positional dependencies between consecutive pods' storage locations.

The integration of time uncertainty situates our work within DRO literature. Unlike classical robust optimization (RO) − which constructs uncertainty sets for parameter realizations and often yields overly conservative solutions (Delage and Mannor, 2010) −

DRO characterizes ambiguity in the underlying probability distributions through data-informed ambiguity sets (Wiesemann et al., 2014). This approach mitigates excessive conservatism while leveraging observational data. However, implementing DRO in multi-stage settings poses significant computational challenges: identifying worst-case distributions within nested ambiguity sets (used to model evolving uncertainty (Xu and Mannor, 2012)) remains tractable primarily for convex problems. Our formulation introduces combinatorial complexity through two coupled integer variable sets governing order assignment and task allocation. To address this, we develop a polynomial-time approximation algorithm with provable suboptimality bounds for the multi-stage DRO problem. This strategy decomposes the optimization at each decision epoch into tractable subproblems while preserving consistency between the coupled order assignment and task allocation decisions. Consequently, our DRO framework captures decision-makers' evolving utilization of distributional knowledge while maintaining feasibility for real-time scheduling.

The DRO approach has been widely used to model uncertainty in various optimization problems. For example, Carlsson and Delage (2013) and Carlsson et al. (2018) applied DRO to service region partitioning problems in the context of the classic traveling salesman problem. In the domain of last-mile delivery, Liu et al. (2020) employed DRO to capture uncertain delivery times in order allocation, while Ulmer et al. (2021) modeled demand and lead time uncertainty using DRO for stochastic dynamic pickup and delivery problems. In contrast to these studies, we establish a novel integrated production-distribution scheduling framework incorporating travel time uncertainty. Moreover, we jointly address the online dispatching of both AMRs and workstations − a complex setting aligned with real-world RMFS constraints. This approach extends DRO methodology to a broader class of tightly coupled production-transportation systems.

3 Model Settings and Problem Overview

3.1 *System Settings and Assumptions*

Consider a RMFS comprising a set of tasks $\mathcal{I} = \{1, 2, \ldots, |\mathcal{I}|\}$ to be completed within the planning horizon, a set of workstations $\mathcal{J} = \{1, 2, \ldots, |\mathcal{J}|\}$ for order processing, and a set of AMRs $\mathcal{K} = \{1, 2, \ldots, |\mathcal{K}|\}$ for pod transportation. Each transportation task performed by

an AMR corresponds to an order processed at a workstation and sequentially comprises: (1) a pickup sub-task(navigating to the pod's storage location), (2) a delivery sub-task(moving the pod to the workstation area), and (3) a return sub-task(returning the pod to the storage area). Given the one-to-one correspondence, we will use task indices to represent orders throughout the subsequent discussion. As our primary focus lies on the joint allocation of tasks and orders to AMRs and workstations, we assume that both tasks and the locations of pods corresponding to these tasks are exogenous.

The detailed operational procedure of the RMFS is as follows: When a task $i \in \mathcal{I}$ is allocated to an AMR $k \in \mathcal{K}$, the AMR picks up and delivers the required pod to a designated workstation $j \in \mathcal{J}$ for processing. If workstation j is occupied upon arrival, the AMR joins a first-come-first-served (FCFS) queue and waits until all preceding tasks at that workstation are completed. Following processing, the AMR returns the pod to storage and subsequently proceeds to its next allocated task $i' \in \mathcal{I}$, repeating the full pickup-delivery-queue-processing-return cycle until all allocated tasks are fulfilled.

A key distinction of our research lies in the explicit modeling of AMR travel times and workstation processing times. Let $\tau_{ii'}$ denote the total travel time between two consecutive tasks i and i' (assigned to the same AMR), and let p_i represent the deterministic processing time of task i at a workstation. Specifically, $\tau_{ii'}$ equals the combined travel time for completing the return requirement of task i and initiating the pickup and the delivery requirements of task i'. Since temporal uncertainties in RMFS primarily stem from AMR congestion, we model $\tau_{ii'}$ as a stochastic parameter. The following assumptions formalize these considerations:

Assumption 3.1 *The AMRs and workstations are homogeneous in capability. That is,*

(i) For any given tasks i and i', the delivery duration $\tau_{ii'}$, which is stochastic, is invariant across different AMR and workstation combinations;

(ii) Similarly, the processing time p_i for task i is invariant among various workstations.

Assumption 3.2 *(i) The travel time of AMRs is bounded and adheres to the triangle inequality, i.e., $\tau_{ii'} + \tau_{i'i''} \leq \tau_{ii''},\ \tau_{ii'} \in [0, \bar{\tau}],\ \forall i, i', i'' \in \mathcal{I}$;*

(ii) The processing time of tasks at workstations is bounded by $p_i \in [\underline{p}, \bar{p}],\ \forall i \in \mathcal{I}$.

Here, Assumption 3.1 posits that all AMRs are of uniform specifications and that work-

stations, being in close proximity, possess identical processing capabilities. This allows us to focus on the relationship between time and task sequencing without the confounding effects of hardware heterogeneity. Assumption 3.2 ensures that travel times satisfy the triangle inequality, while both transport and processing times are bounded to support the subsequent mathematical analysis.

3.2 *Problem Definition and Classification*

For a given set of tasks $\mathcal{I}$, our research aims to minimize the makespan, denoted by $C_{max} = \max\{C_i, \forall i \in \mathcal{I}\}$, where C_i is the completion time of task i. The decision variables include:

(1) Task allocation to AMRs with sequence, denoted by $\boldsymbol{x} = \{x_{ii'}^k \in \{0,1\},\ \forall i, i' \in \mathcal{I},\ k \in \mathcal{K}\}$. Here, $x_{ii'}^k = 1$ signifies that AMR k is sequentially allocated to tasks i and i' without any intervening tasks;

(2) Order assignment to workstations with sequence, denoted by $\boldsymbol{y} = \{y_{ii'}^j \in \{0,1\},\ \forall i, i' \in \mathcal{I},\ j \in \mathcal{J}\}$. Similarly, $y_{ii'}^j = 1$ indicates that workstation j is sequentially assigned to the orders corresponding to tasks i and i' without any intervening orders.

Given the above setup, our research problem can be transformed into a series of well-defined scheduling problems based on the relative sizes of workstations, AMRs, and tasks. To align with the context of classic scheduling studies, we use $s_{ii'}$ to denote the setting time related to tasks i and i', and r_i to represent the release time of task i, where $s_{ii'} = \tau_{ii'}$, $r_i = \tau_{0i'}$ in our scenario. The dispatching problem can then be categorized as follows.

Proposition 3.1 *(i) If $|\mathcal{I}| \leq \min\{|\mathcal{J}|, |\mathcal{K}|\}$, the problem can be solved in polynomial time, and the objective function is $\max_i\{r_i + p_i\}$.*

(ii) If $|\mathcal{K}| \leq \min\{|\mathcal{I}|, |\mathcal{J}|\}$, the problem is equivalent to $P_{|\mathcal{K}|}|s_{ii'}|C_{\max}$, which is an $\mathcal{NP}$-hard problem.

(iii) If $|\mathcal{J}| < |\mathcal{I}| \leq |\mathcal{K}|$, the problem is equivalent to $P_{|\mathcal{J}|}|r_i|C_{\max}$, which is a an $\mathcal{NP}$-hard problem when $|\mathcal{J}| \geq 2$. In addition, the Longest Processing Time (LPT) Principle provides a $3/2$ approximation algorithm.

(iv) If $|\mathcal{J}| < |\mathcal{K}| < |\mathcal{I}|$, the problem is equivalent to a two-stage flexible flow shop problem $F(|\mathcal{K}|, |\mathcal{J}|)|s_{ii'}, block|C_{\max}$, which is also an $\mathcal{NP}$-hard problem.

Proposition 3.1 categorizes the transformed scheduling problem according to various

parameter combinations. Following Michael (1995), we use P_m to represent the scenario with m machines in parallel, and $F(m, n)$ to denote the two-stage flexible flow shop with m and n parallel machines at the two stages, respectively. Case (i) suggests that the number of tasks is smaller than either the number of AMRs or workstations. Consequently, the corresponding problem can be simplified to a more manageable problem solvable in polynomial time. Cases (ii) and (iii) address scenarios with a limited number and a large number of AMRs, respectively, and both can be transformed into well-studied $\mathcal{NP}$-hard problems. Specifically, Case (ii) is analogous to a parallel machine scheduling problem with $|K|$ machines, with the constraints on the sequence-dependent setup times of $s_{ii'}$. Similarly, Case (iii) corresponds to a parallel machine scheduling problem with $|J|$ machines and respective release times of r_i. Furthermore, these two cases can be transformed into several classic scheduling and routing problems under specific scenarios. For example, when $|\mathcal{K}| = 1$, Case (ii) can be simplified to a Traveling Salesman Problem (TSP). Similarly, when $2 \leq |\mathcal{J}| < |\mathcal{I}| \leq |\mathcal{K}|$, Case (iii) encompasses the following two specific scenarios:

(1) When $r_i = r,\ \forall i \in \mathcal{I}$, the problem is equivalent to minimizing $P_{|\mathcal{J}|}||C_{\max}$. This problem has been studied by Graham (1969), showing that the LPT Principle is a 4/3 approximation algorithm.

(2) When $p_i = p,\ \forall i \in \mathcal{I}$, the problem is a $\mathcal{P}$-problem, and can be optimized via list scheduling (LS) policy based on the set of all available tasks.

Case (iv) in Proposition 3.1 presents the most complex scenario with fewer workstations and a large number of tasks. This is equivalent to a two-stage flexible shop with $|K|$ and $|J|$ parallel machines at the two stages, subject to the setup times of $s_{ii'}$ and a critical "block" constraint: After completing its portion of a task, the machine (AMR) remains occupied until the task finishes all subsequent operations. The remainder of this research focuses exclusively on this operationally complex yet prevalent RMFS scenario, which remains understudied in scheduling literature.

3.3 *Mathematical Formulation of the Integrated Model*

To address the integrated dispatching problem, we present a stochastic programming model that accounts for uncertainties in travel times. In addition to $\boldsymbol{x}$ and $\boldsymbol{y}$, we introduce two intermediate variables: (1) $\boldsymbol{u} = \{u_i, \forall i \in \mathcal{I}\}$, where u_i denotes the arrival time of the AMR

delivering task i to the corresponding workstation; and (2) $\boldsymbol{v} = \{v_i, \forall i \in \mathcal{I}\}$, in which v_i represents the completion time of task i's preceding task at the workstation. Let $i = 0$ and $i = |\mathcal{I}| + 1$ denote virtual dummy tasks representing the initialization and termination of operations for both AMRs and workstations.. Let $\pi \in \Pi$ represent the optimal policy to make, where Π is the set of all non-anticipative policies (that is, policies that only depend on the observed history). Then the mathematical formulation is given as follows:

$$\min_{\pi \in \Pi} \ \mathbb{E}^{\tau} \{C_{\max}\} \tag{1a}$$

$$s.t. \ \ C_{\max} \geq C_i, \forall i \in \mathcal{I}, \tag{1b}$$

$$\sum_{i' \in \mathcal{I}} x_{0i'}^k = 1, \forall k \in \mathcal{K}, \tag{1c}$$

$$\sum_{i' \in \mathcal{I} \backslash \{i\}} x_{ii'}^k - \sum_{i'' \in \mathcal{I} \backslash \{i\}} x_{i''i}^k = 0, \forall k \in \mathcal{K}, \forall i \in \mathcal{I}, \tag{1d}$$

$$\sum_{i \in \mathcal{I}} x_{i,|\mathcal{I}|+1}^k = 1, \forall k \in \mathcal{K}, \tag{1e}$$

$$\sum_{k \in \mathcal{K}} \sum_{i \in \mathcal{I} \cup \{0\}} x_{ii'}^k = 1, \forall i' \in \mathcal{I} \backslash \{i\}, \tag{1f}$$

$$\sum_{i' \in \mathcal{I}} y_{0i'}^j = 1, \forall j \in \mathcal{J}, \tag{1g}$$

$$\sum_{i' \in \mathcal{I} \backslash \{i\}} y_{ii'}^j - \sum_{i'' \in \mathcal{I} \backslash \{i\}} y_{i''i}^j = 0, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}, \tag{1h}$$

$$\sum_{i \in \mathcal{I}} y_{i,|\mathcal{I}|+1}^j = 1, \forall j \in \mathcal{J}, \tag{1i}$$

$$\sum_{j \in \mathcal{J}} \sum_{i \in \mathcal{I} \cup \{0\}} y_{ii'}^j = 1, \forall i' \in \mathcal{I} \backslash \{i\}, \tag{1j}$$

$$(C_i + \tau_{ii'} - u_{i'}) x_{ii'}^k = 0, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall k \in \mathcal{K}, \tag{1k}$$

$$(C_i - v_{i'}) y_{ii'}^j = 0, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall j \in \mathcal{J}, \tag{1l}$$

$$C_i = \max\{u_i, v_i\} + p_i, \forall i \in \mathcal{I}, \tag{1m}$$

$$(u_i - u_{i'}) y_{ii'}^j \leq 0, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall j \in \mathcal{J}, \tag{1n}$$

$$x_{ii'}^k \in \{0,1\}, y_{ii'}^j \in \{0,1\}, \forall k \in \mathcal{K}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \tag{1o}$$

$$C_i \geq 0, \forall i \in \mathcal{I}. \tag{1p}$$

Here, the objective function (1a) aims to minimize the makespan, which is defined as the maximum completion time across all tasks in Constraint (1b). Conditions (1c) to (1f) are constraints related to AMRs. Specifically, Constraint (1c) requires each AMR to depart

from the docking position and start to execute tasks exactly once. Constraint (1d) is a flow conservation for sequential task execution. Constraint (1e) requires each AMR to complete all assigned tasks and return to the docking position exactly once. Constraint (1f) ensures that each task is assigned to exactly one AMR. Similarly, constraints (1g) to (1j) enforce the sequential processing of tasks at workstations, mirroring the structure of the AMR-related constraints. Constraints (1k) to (1n) characterize the temporal relationships among tasks, and their linearization is provided in appendix A.2. Specifically, Constraint (1k) ensures that for two consecutive tasks i and i' performed by the same AMR, the AMR's arrival time of i' ($u_{i'}$) at the workstation equals the completion time of i (C_i) plus the transit time $\tau_{ii'}$. And v_i is defined in Constraint (1l) as the completion time of task i's preceding task at the workstation. Completion time C_i is defined in Constraint (1m) as the maximum of AMR arrival time u_i and workstation release time v_i, plus processing time p_i. Constraint (1n) enforces a FCFS queue at each workstation, ensuring that tasks are processed in the same sequence as they arrive. Finally, Constraint (1o) stipulates that the decision variables are binary, and Constraint (1p) ensures the non-negativity of task completion times.

The complexity of model 1 is evident, even without considering the travel time uncertainty. The procedures for completing tasks in our context essentially resemble the classic Vehicle Routing Problem (VRP). However, our problem jointly considers the order and task allocation for both AMRs and workstations, thereby obtaining two coupled sets of VRP-based constraints. This substantially amplifies the complexity of the mathematical formulation and the challenge of solving the problem.

4 Dynamic Programming Model

This section extends the stochastic programming Model 1 into a solvable dynamic programming model for multi-period planning, which lays the groundwork for further online dispatching strategies.

4.1 *Problem Simplification via Lagrange Relaxation*

A key limitation of the original model lies in its objective function, which is designed to minimize the system-wide makespan ($C_{\max}$). This formulation is ill-suited for online

optimization because the value of $C_{\max}$ can only be determined after all tasks have been fully allocated − a critical constraint that complicates the real-time analysis of how specific task allocation decisions affect system performance. To address this, we propose replacing the original objective $C_{\max}$ with refined, component-specific objectives: workstation-level makespan ($C^j_{\max}, \forall j$) and AMR-level makespan ($C^k_{\max}, \forall k$). By decoupling the system-level completion time into sub-objectives, we develop an approximate objective function that better aligns with the dynamic, incremental nature of online optimization strategies. Additionally, we demonstrate that the bounded gap between these sub-objectives and the original $C_{\max}$ strengthens theoretical global consistency.

4.1.1 *Preliminary: Mathematical Properties of the Optimal Solutions*

We begin by analyzing the gaps between the system makespan and the completion times of the workstations and AMRs under the optimal strategy. Specifically, we first introduce a valid inequality in Proposition 4.1, enabling us to calculate the optimal makespans for the system and its components. In the subsequent comparison stage, we conduct a comprehensive analysis at both the individual and aggregated levels. For analytical convenience, we primarily focus on the deterministic case, assuming that travel times are known in advance.

Proposition 4.1 *The optimal solutions of Model 1 exist such that the last task processed at each workstation corresponds to the final task assigned to the respective AMR. The valid inequality corresponding to this condition can be expressed as follows:*

$$y^j_{i,|\mathcal{I}|+1} \leq \sum_{k \in \mathcal{K}} x^k_{i,|\mathcal{I}|+1}, \forall i \in \mathcal{I}, \forall j \in \mathcal{J}. \tag{2}$$

Proposition 4.1 establishes that optimal solutions satisfying constraint (2) must exist. The condition for optimality is quite intuitive: it implies that the last task at every workstation corresponds to an AMR that does not need to service subsequent tasks. Otherwise, we can always improve the solution by transferring all subsequent tasks for that AMR to the current workstation.

Based on Proposition 4.1, we subsequently analyze the maximum gap between the objective function $C_{\max}$ and the completion time of each workstation $C^j_{\max}$, as well as the completion time of each AMR $C^k_{\max}$. Specifically, Propositions 4.2 and 4.3 provide sufficient

conditions for the existence of an optimal solution, while Propositions 4.4 and 4.5 present conditions that all optimal solutions must satisfy, which are derived from inequality (2).

Proposition 4.2 *There exist optimal solutions to Model 1 such that*

$$C_{\max} - \min_{j \in \mathcal{J}} C^j_{\max} \leq \bar{\tau} + \bar{p}. \tag{3}$$

Proposition 4.2 asserts the existence of optimal solutions where the bounded gap between the objective $C_{\max}$ and the minimum workstation makespan $\min_{j \in \mathcal{J}} C^j_{\max}$ is less than $\bar{\tau} + \bar{p}$ − the per-task upper limit of transport and processing times. Notably, since this bound is independent of the number of allocated tasks ($|\mathcal{I}|$), the ratio of the bounded gap to the objective $C_{\max}$ diminishes as $|\mathcal{I}|$ grows.

Proposition 4.3 *There exist optimal solutions to Model 1 that satisfy at least one of the following two bounds:*

(i) AMR-related bound:

$$C_{\max} - \min_{k \in \mathcal{K}} C^k_{\max} \leq \bar{\tau} + \bar{p}; \tag{4}$$

(ii) Workstation-related bound:

$$C_{\max} - \min_{j \in \mathcal{J}} C^j_{\max} \leq \bar{p}. \tag{5}$$

Proposition 4.3 further delineates the gap between the objective $C_{\max}$ and the minimum completion time of AMRs $\min_{k \in \mathcal{K}} C^k_{\max}$, in addition to the gap with $\min_{j \in \mathcal{J}} C^j_{\max}$ under condition (2). Here, either the gap between $C_{\max}$ and $\min_{k \in \mathcal{K}} C^k_{\max}$ is less than the maximum sum of a task's travel time and processing time $\bar{\tau} + \bar{p}$, or the gap between $C_{\max}$ and $\min_{j \in \mathcal{J}} C^j_{\max}$ is less than the upper bound of the operating time of a task $\bar{p}$. Compared with Proposition 4.2, Proposition 4.3 further suggests that if condition 4 does not hold, then the upper bound of $C_{\max} - \min_{j \in \mathcal{J}} C^j_{\max}$ can be further reduced by $\bar{\tau}$.

Proposition 4.4 *Under the valid inequality* (2)*, the optimal solutions to Model 1 satisfy:*

$$|\mathcal{J}| C_{\max} - \sum_{j \in \mathcal{J}} C^j_{\max} \leq (|\mathcal{J}| - 1)(\bar{\tau} + \bar{p}). \tag{6}$$

Proposition 4.4 further refines the upper bound on the gap between the objective $C_{\max}$ and the average completion time of workstations. Specifically, the average-level gap $C_{\max} -$

$\frac{1}{|\mathcal{J}|}\sum_{j\in\mathcal{J}} C^j_{\max} \leq \frac{|\mathcal{J}|-1}{|\mathcal{J}|}(\bar{\tau}+\bar{p})$. Similar to the results in Proposition 4.2, the aggregate-level bound (6) is independent of $|\mathcal{I}|$; thus, it can be considered a tight bound when dealing with a large number of tasks. Moreover, the average-level bound is slightly reduced compared to the individual-level bound, a result that aligns with expectations.

Proposition 4.5 *Under the valid inequality* (2)*, the optimal solutions to Model 1 satisfy at least one of the following two bounds:*

(i) AMR-related bound:

$$|\mathcal{K}|C_{\max} - \sum_{k\in\mathcal{K}} C^k_{\max} \leq (|\mathcal{K}|-1)(\bar{\tau}+\bar{p}),\ \forall k \in \mathcal{K}; \tag{7}$$

(ii) Workstation-related bound:

$$|\mathcal{J}|C_{\max} - \sum_{j\in\mathcal{J}} C^j_{\max} \leq (|\mathcal{J}|-1)\bar{p},\ \forall j \in \mathcal{J}. \tag{8}$$

Similar to Proposition 4.3, Proposition 4.5 further characterizes the gap between the objective function $C_{\max}$ and the average completion time of AMRs. Under the valid inequality (2), either the gap between $C_{\max}$ and the average completion time of AMRs ($\frac{1}{|\mathcal{K}|}\sum_{k\in\mathcal{K}} C^k_{\max}$) is less than $\frac{|\mathcal{K}|-1}{|\mathcal{K}|}(\bar{\tau}+\bar{p})$, or the gap between $C_{\max}$ and the average completion time of workstations ($\frac{1}{|\mathcal{J}|}\sum_{j\in\mathcal{J}} C^j_{\max}$) is less than $\frac{|\mathcal{J}|-1}{|\mathcal{J}|}\bar{p}$. Compared to Proposition 4.4, Proposition 4.5 further implies that if condition 4 does not hold, then the upper bound $C_{\max} - \frac{\sum_{j\in\mathcal{J}} C^j_{\max}}{|\mathcal{J}|}$ can be further reduced by $\frac{(|\mathcal{J}|-1)\bar{\tau}}{|\mathcal{J}|}$.

4.1.2 *Approximated Objective Function via Lagrange Relaxation*

The theoretical results confirm that the bounded gap between the system-wide makespan and the completion time of components is not significant. Consequently, we construct an approximate objective function based on $C^j_{\max}$ and $C^k_{\max}$ using the Lagrange relaxation method. Specifically, given that constraint (1b) is equivalent to the following two conditions:

$$C_{\max} \geq \sum_{i\in\mathcal{I}} C_i x^k_{i,|\mathcal{I}|+1}, \forall k \in \mathcal{K},\quad C_{\max} \geq \sum_{i\in\mathcal{I}} C_i y^j_{i,|\mathcal{I}|+1}, \forall j \in \mathcal{J},$$

we can formulate the Lagrange relaxation problem with the following objective function:

$$C_{\max} - \sum_{k\in\mathcal{K}} \lambda_k (C_{\max} - \sum_{i\in\mathcal{I}} C_i x^k_{i,|\mathcal{I}|+1}) - \sum_{j\in\mathcal{J}} \mu_j (C_{\max} - \sum_{i\in\mathcal{I}} C_i y^j_{i,|\mathcal{I}|+1}). \tag{9}$$

Here, λ_k and μ_j are non-negative Lagrange multipliers with $\sum_{k\in\mathcal{K}}\lambda_k + \sum_{j\in\mathcal{J}}\mu_j = 1$. As shown in Section A.9, the objective function (9) is equivalent to

$$\sum_{k\in\mathcal{K}}\lambda_k \sum_{i\in\mathcal{I}\cup\{0\}}\sum_{i'\in\mathcal{I}}((v_{i'}-u_{i'})^+ + p_{i'} + \tau_{ii'})x^k_{ii'} + \sum_{j\in\mathcal{J}}\mu_j \sum_{i\in\mathcal{I}\cup\{0\}}\sum_{i'\in\mathcal{I}}((u_{i'}-v_{i'})^+ + p_{i'})y^j_{ii'}. \quad (10)$$

As defined in Section 3.3, u_i is the arrival time of task i at the corresponding workstation, and v_i represents the completion time of the previous task at the same workstation. The Lagrange relaxation problem provides a lower bound for the original problem.

Lemma 4.1 *The lower bounds obtained with equal Lagrange multipliers of the same type, i.e., $\lambda_k = \lambda, \forall k \in \mathcal{K}$, $\mu_j = \mu, \forall j \in \mathcal{J}$, are larger than those with unequal multipliers.*

Lemma 4.1 provides the optimum of Lagrange relaxation problems, thus offering the tightest lower bound of the original problem among all Lagrange relaxation variants. Therefore, we set $\lambda_k = \lambda = \frac{\gamma}{|\mathcal{K}|}, \forall k \in \mathcal{K}$, $\mu_j = \mu = \frac{1-\gamma}{|\mathcal{J}|}, \forall j \in \mathcal{J}$, and the objective function (10) can be further transformed to

$$\sum_{i'\in\mathcal{I}} \underbrace{\frac{\gamma}{|\mathcal{K}|}(v_{i'}-u_{i'})^+}_{(i)} + \underbrace{\frac{1-\gamma}{|\mathcal{J}|}(u_{i'}-v_{i'})^+}_{(ii)} + \underbrace{(\frac{\gamma}{|\mathcal{K}|}+\frac{1-\gamma}{|\mathcal{J}|})p_{i'}}_{(iii)} + \underbrace{\frac{\gamma}{|\mathcal{K}|}\sum_{i\in\mathcal{I}\cup\{0\}}\tau_{ii'}\sum_{k\in\mathcal{K}}x^k_{ii'}}_{(iv)}. \quad (11)$$

The transformation details are given in Section A.9. Here, sub-objective (iii) is irrelevant to the decisions and can be considered as a fixed item. Sub-objective (i) is the sum of waiting time of tasks at workstations, while sub-objective (ii) is the sum of workstations' idle time. Sub-objective (iv) is the total travel time on AMRs. In summary, the modified objective function of the Lagrange relaxation problem decomposes the original objective of total system makespan into the sum of task-specific sub-objectives. Consequently, during subsequent online allocation, we can compute the reward for each task, which supports the design of online optimization strategies.

4.2 *Dynamic Programming Model*

Building upon the decomposed sub-objectives of each task, we develop a dynamic programming model to facilitate the subsequent online dispatching strategy. We define the time period as t and the starting time of period t as η_t. Here, we assume $\eta_t = t\Delta\eta$, with $\Delta\eta$ representing a fixed time slot. In addition, we introduce the following additional decision

variables: (1) The set of unserved tasks in period t, denoted by $\mathcal{A}_t$, where $|\mathcal{A}_t| \leq |\mathcal{I}|$; (2) The set of tasks being executed by AMRs in period t, denoted by $\mathcal{B}_t$, where $|\mathcal{B}_t| \leq |\mathcal{K}|$; (3) The set of tasks that are allocated to AMRs in period t, denoted by $\Delta\mathcal{A}_t \subseteq \mathcal{A}_t$; and (4) The set of tasks that are finished by AMRs in period t, denoted by $\Delta\mathcal{B}_t \subseteq \mathcal{B}_t$. (5) The starting time of task i, denoted by $w_i,\ \forall i \in \mathcal{I}$.

Then the Bellman equation for our dynamic programming model is as follows:

$$H_t(\mathcal{A}_t, \mathcal{B}_t) = \min_{\Delta\mathcal{A}_t \subseteq \mathcal{A}_t || \Delta\mathcal{A}_t| \leq |\Delta\mathcal{B}_t|} \sum_{i \in \Delta\mathcal{B}_t} (\frac{1-\gamma}{|\mathcal{J}|}(u_i - v_i)^+ + \frac{\gamma}{|\mathcal{K}|}(v_i - u_i)^+ + \frac{\gamma}{|\mathcal{K}|}(u_i - w_i)) + \mathbb{E}^\tau\{H_{t+1}(\mathcal{A}_{t+1}, \mathcal{B}_{t+1})\} \tag{12}$$

where $\Delta\mathcal{B}_t = \{i \in \mathcal{B}_t | \max\{u_i, v_i\} \in (\eta_{t-1}, \eta_t]\}$. The transition function is given by: $(\mathcal{A}_{t+1}, \mathcal{B}_{t+1}) = (\mathcal{A}_t \backslash \Delta\mathcal{A}_t, \mathcal{B}_t \cup \Delta\mathcal{A}_t \backslash \Delta\mathcal{B}_t\})$, and the boundary condition is $H_t(\emptyset, \emptyset) = 0, \forall t$.

Unfortunately, the curse of dimensionality renders the direct solution of this dynamic programming model infeasible, primarily due to the large task volume and travel time uncertainty. To address this, we propose an ADP model adopting myopic strategies – a common practice in online scheduling problems. Specifically, the reward function is computed solely based on currently assigned tasks, and only the impacts of tasks assigned in the current and previous periods are considered. Future unassigned tasks are explicitly excluded from the current optimization.

To develop an effective ADP model, we first reformulate the original dynamic programming model into an equivalent form. This equivalent model exhibits more forward-looking system behavior under the myopic policy. We define $v_i(\mathcal{S})$ as the time required for the workstation corresponding to task i to complete the previous tasks when only the tasks in set $\mathcal{S}$ are available. It is straightforward that $v_i(\cdot)$ is a monotonic function, i.e., $v_i(\mathcal{S}_1 \cup \mathcal{S}_2) \geq v_i(\mathcal{S}_1)$, $v_i(\mathcal{S}_1 \cup \mathcal{S}_2) \geq v_i(\mathcal{S}_2)$. Then, the previous Bellman's equation (12) can be reformulated as:

$$\tilde{H}_t(\mathcal{A}_t, \mathcal{B}_t) = \min_{\Delta\mathcal{A}_t \subseteq \mathcal{A}_t || \Delta\mathcal{A}_t| \leq |\Delta\mathcal{B}_t|} \mathbb{E}^\tau\{\tilde{G}(\Delta\mathcal{A}_t, \mathcal{B}_t) + \tilde{H}_{t+1}(\mathcal{A}_{t+1}, \mathcal{B}_{t+1})\}, \tag{13}$$

where $\Delta\mathcal{B}_t = \{i \in \mathcal{B}_t | \max\{u_i, v_i\} \in (\eta_{t-1}, \eta_t]\}$, $\tilde{G}(\Delta\mathcal{A}_t, \mathcal{B}_t) = G_1(\Delta\mathcal{A}_t, \mathcal{B}_t) + G_2(\Delta\mathcal{A}_t, \mathcal{B}_t)$, $\tilde{G}_1(\Delta\mathcal{A}_t, \mathcal{B}_t) = \sum_{i \in \mathcal{B}_t} \frac{1-\gamma}{|\mathcal{J}|}[(u_i - v_i(\mathcal{B}_t \cup \Delta\mathcal{A}_t))^+ - (u_i - v_i(\mathcal{B}_t))^+] + \frac{\gamma}{|\mathcal{K}|}[(v_i(\mathcal{B}_t \cup \Delta\mathcal{A}_t) - u_i)^+ - (v_i(\mathcal{B}_t) - u_i)^+]$, and $\tilde{G}_2(\Delta\mathcal{A}_t, \mathcal{B}_t) = \sum_{i \in \Delta\mathcal{A}_t} \frac{1-\gamma}{|\mathcal{J}|}(u_i - v_i(\mathcal{B}_t \cup \Delta\mathcal{A}_t))^+ + \frac{\gamma}{|\mathcal{K}|}(v_i(\mathcal{B}_t \cup \Delta\mathcal{A}_t) - u_i)^+ + \frac{\gamma}{|\mathcal{K}|}(u_i - w_i)$. And both the transition function and boundary condition remain

unchanged. Under the Myopic strategy, we have:

$$\begin{aligned}\tilde{G}(\Delta\mathcal{A}_t, \mathcal{B}_t) &= \frac{1-\gamma}{|\mathcal{J}|}\sum_{j\in\mathcal{J}}\Big(C^j_{\max}(\mathcal{B}_t\cup\Delta\mathcal{A}_t) - C^j_{\max}(\mathcal{B}_t)\Big) \\ &+ \frac{\gamma}{|\mathcal{K}|}\left(\sum_{i\in\mathcal{B}_t\cup\Delta\mathcal{A}_t} C_i(\mathcal{B}_t\cup\Delta\mathcal{A}_t) - \sum_{i\in\mathcal{B}_t} C_i(\mathcal{B}_t)\right) - (\frac{1-\gamma}{|\mathcal{J}|}+\frac{\gamma}{|\mathcal{K}|})\sum_{i\in\Delta\mathcal{A}_t} p_i,\end{aligned}$$

where, $C^j_{\max}(\mathcal{S})$ represents the completion time of workstation j with the set of tasks $\mathcal{S}$, and $C_i(\mathcal{S})$ is the completion time of task $i \in \mathcal{S}$. Let

$$G(\mathcal{S}) = \frac{1-\gamma}{|\mathcal{J}|}\sum_{j\in\mathcal{J}} C^j_{\max}(\mathcal{S}) + \frac{\gamma}{|\mathcal{K}|}\sum_{i\in\mathcal{S}} C_i(\mathcal{S}) - (\frac{1-\gamma}{|\mathcal{J}|}+\frac{\gamma}{|\mathcal{K}|})\sum_{i\in\mathcal{S}} p_i, \tag{14}$$

we have $\tilde{G}(\Delta\mathcal{A}_t, \mathcal{B}_t) = G(\mathcal{B}_t\cup\Delta\mathcal{A}_t) - G(\mathcal{B}_t)$. Thus, the ADP model is given by

$$\hat{H}_t(\mathcal{A}_t, \mathcal{B}_t) = \mathbb{E}^\tau[\hat{H}_{t+1}(\mathcal{A}_{t+1}, \mathcal{B}_{t+1}) - G(\mathcal{B}_t)] + \min_{\Delta\mathcal{A}_t\subseteq\mathcal{A}_t||\Delta\mathcal{A}_t|\le|\Delta\mathcal{B}_t|} \mathbb{E}^\tau[G(\mathcal{B}_t\cup\Delta\mathcal{A}_t)]. \tag{15}$$

The ADP model (15) reveals that at each decision epoch, the problem reduces to an optimization of function $G(\cdot)$, i.e., $\min_{\Delta\mathcal{A}_t\subseteq\mathcal{A}_t||\Delta\mathcal{A}_t|\le|\Delta\mathcal{B}_t|}\mathbb{E}^\tau[G(\mathcal{B}_t\cup\Delta\mathcal{A}_t)]$.

4.3 *Sub-Optimality Induced by the FCFS Rule*

We conclude this section by evaluating the impact of the FCFS rule within the approximated problem, using the optimal schedule as a benchmark. Since FCFS is a conventional practice in RMFS, its inherent system inefficiencies are inevitable. Given that our focus is on tasks assigned within the current period, the decision at each decision epoch simplifies to assigning the next task to each AMR as it arrives at the workstation. In this context, the travel time for the next task depends only on the specific characteristics of that task, and the objective function is $\frac{\gamma}{|\mathcal{K}|}\sum C_i + \frac{1-\gamma}{|\mathcal{J}|}C_{\max}$ according to the ADP model (15). Therefore, for each workstation, this structure constitutes a single-machine scheduling problem $1|r_i|\frac{\gamma}{|\mathcal{K}|}\sum C_i + \frac{1-\gamma}{|\mathcal{J}|}C_{\max}$, where the release time r_i equals task i's arrival time at the workstation, and the FCFS rule is mathematically equivalent to the earliest release date (ERD) rule in the setting of scheduling literature. Without loss of generality, we further assume no prioritization exists for simultaneous task arrivals ($r_i = r_{i'}$).

Proposition 4.6 *For a single-machine scheduling problem* $1|r_i|\frac{\gamma}{|\mathcal{K}|}\sum C_i + \frac{1-\gamma}{|\mathcal{J}|}C_{\max}$*, the maximum gap between the optimum and the FCFS rule is* $\frac{\gamma}{|\mathcal{K}|}\lceil\frac{|\mathcal{I}_j|}{2}\rceil\lfloor\frac{|\mathcal{I}_j|}{2}\rfloor(\bar{p}-\underline{p})$.

With the above settings, the gap between the FCFS rule and the optimum is given in Proposition 4.6. Here, $|\mathcal{I}_j|$ denotes the number of tasks for workstation $j \in \mathcal{J}$, and $\frac{\gamma}{|\mathcal{K}|}$ is the Lagrangian multiplier. Indeed, the ERD rule achieves exact optimality for the single-machine scheduling problem $1|r_i|C_{\max}$. Thus, the observed optimality gap under our objective function $\frac{\gamma}{|\mathcal{K}|}\sum C_i + \frac{1-\gamma}{|\mathcal{J}|}C_{\max}$ stems from the $\sum C_i$ component, where the FCFS rule incurs suboptimality proportional to task processing time variance $(\bar{p} - \underline{p})$.

Proposition 4.7 *For a single-machine scheduling problem* $1|r_i|\frac{\gamma}{|\mathcal{K}|}\sum C_i + \frac{1-\gamma}{|\mathcal{J}|}C_{\max}$ *with buffer size* $|\bar{\mathcal{K}}_j|$*, we have:*

(i) If $|\bar{\mathcal{K}}_j| \geq \lfloor \frac{|\mathcal{I}_j|}{2} \rfloor$*, the maximum gap between the optimal schedule and the FCFS rule equals* $\frac{\gamma}{|\mathcal{K}|}\lceil \frac{|\mathcal{I}_j|}{2} \rceil \lfloor \frac{|\mathcal{I}_j|}{2} \rfloor (\bar{p} - \underline{p})$*;*

(ii) Otherwise, the maximum gap equals $\frac{\gamma}{|\mathcal{K}|}|\bar{\mathcal{K}}_j|(|\mathcal{I}_j| - |\bar{\mathcal{K}}_j|)(\bar{p} - \underline{p})$*.*

In practice, there is typically an upper limit to the number of AMRs assigned to each workstation for load balancing. Let $|\bar{\mathcal{K}}_j|$ denote the upper limit for workstation $j \in \mathcal{J}$. The problem then becomes equivalent to a single-machine scheduling problem with a buffer size of $|\bar{\mathcal{K}}_j|$. The gap between the FCFS rule and the optimal sequence is detailed in Proposition 4.7. In case 1, where the total number of tasks on each workstation ($|\mathcal{I}_j|$) is relatively small, the gap is approximately quadratic with respect to $|\mathcal{I}_j|$. Meanwhile, in case 2, the gap is linear with $|\mathcal{I}_j|$, suggesting that the number of tasks has less significant marginal effects on the gap when the total number of tasks gets larger. In Case 1, where the task load per workstation ($|\mathcal{I}_j|$) is relatively small, the optimality gap scales quadratically with $|\mathcal{I}_j|$. Conversely, in Case 2 with larger task volumes, the gap exhibits linear scaling with $|\mathcal{I}_j|$. This transition reveals diminishing marginal impact of task volume on the per-task scheduling loss as the total number of tasks ($|\mathcal{I}|$) increases.

5 DRO under Travel Time Uncertainty

Based on the ADP model, we incorporate DRO techniques to capture the time uncertainty. We first clarify two key motivations for adopting DRO. First, DRO enhances operational stability in RMFS by minimizing system efficiency fluctuations. As this approach optimizes against the worst-case probability distribution, it improves robustness against extreme scenarios and mitigates efficiency risks. Second, accurate estimation of travel times is inher-

ently challenging in this system. Pre-task transport durations cannot be reliably predicted from historical data due to dynamic factors such as task types, pickup/delivery locations, and real-time warehouse traffic conditions.

In subsequent numerical experiments, we analyze travel time distributions in RMFS and demonstrate that point estimates frequently deviate significantly from actual values, while interval estimation captures a broader range of real-world variations. Therefore, following the distributionally robust MDP framework established in Xu and Mannor (2012), we employ a sequence of nested ambiguity sets.

5.1 *Formulation of the Ambiguity Sets*

We first define the nested ambiguity sets for the travel time of tasks. To achieve workload equilibrium among distributed resources and ensure problem tractability, we adopt the following dispatching strategy: At each decision epoch, at most one task is allocated to each workstation and idle AMR. Mathematically, we use the type tuple $\phi = \langle i, j, k \rangle \in \Phi$ to represent the specific decision of assigning the task i to the AMR k and transporting it to the workstation j for processing. The ambiguity set of type ϕ, determined by the current state of task i, workstation j and AMR k, describes the uncertainty in the distribution of travel times for unprocessed tasks assigned to workstation j. As discussed in Section 4.3, our research problem essentially equates to a single-machine scheduling problem with release times denoted by $\boldsymbol{r} = \{r_i, \forall i \in \mathcal{I}\}$. As a result, we use $\boldsymbol{r}_\phi$ to represent the release time vector for all tasks currently assigned to the workstation associated with ϕ.

We employ a series of N nested ambiguity sets $\mathscr{R}^1 \subseteq \mathscr{R}^2 \subseteq \cdots \subseteq \mathscr{R}^N$ with associated confidence levels $0 \leq \sigma_1 \leq \sigma_2 \leq \cdots \leq \sigma_N = 1$. where σ_n quantifies the probabilistic guarantee for set $\mathscr{R}^n$ of the unknown parameters (release time vectors $\boldsymbol{r}^n_\phi, \forall \phi$). For ease of notation, we define $\sigma_0 = 0$. Crucially, when $N = 1$, this framework reduces to conventional RO with $\mathscr{R}^1$ constituting the deterministic uncertainty set.

Let ψ represent a specific scenario encompassing the allocation of all new tasks at a decision epoch, and Ψ the universe of all such scenarios. The ambiguity set $\mathscr{F}^\Psi(\sigma_1, \ldots, \sigma_N)$ is structured as follows:

$$\mathscr{F}^\Psi(\sigma_1, \ldots, \sigma_N) \quad = \quad \left\{ f | f = \bigotimes_{\psi \in \Psi} f^\psi; f^\psi \in \mathscr{F}^\psi(\sigma_1, \ldots, \sigma_N), \forall \psi \in \Psi \right\} \tag{16}$$

$$\mathscr{F}^{\psi}(\sigma_1,\ldots,\sigma_N) = \left\{f^{\psi} | f^{\psi}_{\phi} = f_{\phi}; f_{\phi} \in \mathscr{F}_{\phi}(\sigma_1,\ldots,\sigma_N), \forall \phi \in \Phi_{\psi}\right\} \tag{17}$$

$$\mathscr{F}_{\phi}(\sigma_1,\ldots,\sigma_N) = \left\{f_{\phi} | f_{\phi}(\mathscr{R}^N_{\phi}) = 1; f_{\phi}(\mathscr{R}^n_{\phi}) \geq \sigma_n, n = 1,\ldots,N-1\right\}. \tag{18}$$

In Eq. (16), $\mathscr{F}^{\psi}(\sigma_1,\ldots,\sigma_N)$ represents the ambiguity set for a specific scenario ψ, and the condition $f = \bigotimes_{\psi\in\Psi} f^{\psi}$ indicates that the parameters for different scenarios are independent, where $\bigotimes$ denotes the Cartesian product. Next, in Eq. (17), we introduce $\Phi_{\psi} \subseteq \Phi$ to represent the set of all allocation tuples ϕ under a specific scenario ψ, and $\mathscr{F}_{\phi}(\sigma_1,\ldots,\sigma_N)$ is the marginal ambiguity set for a specific $\psi \in \Psi$. As different type tuples ϕ correspond to different workstations, we only need to capture the marginal distribution in the formulation, rather than the joint distribution. The condition $f^{\psi}_{\phi} = f_{\phi}$ indicates that f_{ϕ} is the marginal distribution f^{ψ}_{ϕ} of f^{ψ} over the type ϕ. Finally, Eq. (18) provides the specific conditions for a given type tuple ϕ. Specifically, given that $\boldsymbol{r}_{\phi}$ is the release time vector for all tasks currently assigned to the workstation associated with ϕ, the condition $f_{\phi}(\mathscr{R}^N_{\phi}) = 1$ implies that the unknown parameter $\boldsymbol{r}_{\phi}$ is restricted to the outermost ambiguity set, and the condition $f_{\phi}(\mathscr{R}^n_{\phi}) \geq \sigma_n$ implies that $\boldsymbol{r}_{\phi} \in \mathscr{R}^n_{\phi}$ holds with a probability of at least σ_n.

5.2 *Distributionally Robust Counterpart*

Given the defined ambiguity set $\mathscr{F}^{\Psi}(\sigma_1,\ldots,\sigma_N)$ characterizing scenario-based uncertainties, we formulate the following distributionally robust counterpart to the ADP model (15).

$$\min_{\boldsymbol{s}} \sup_{f^{\Psi}\in\mathscr{F}^{\Psi}(\sigma_1,\ldots,\sigma_N)} \mathbb{E}^{\boldsymbol{r}} \left\{ \sum_{\psi\in\Psi} \sum_{\phi\in\Phi_{\psi}} \max_{\boldsymbol{y}^{\phi}} \min_{\boldsymbol{C}^{\phi}} g^{\phi}\left(\boldsymbol{r}_{\phi}, \boldsymbol{y}^{\phi}, \boldsymbol{C}^{\phi}\right) s_{\psi} \right\} \tag{19a}$$

$$s.t. \quad \sum_{\psi\in\Psi} s_{\psi} = 1, \tag{19b}$$

$$\boldsymbol{y}_{\phi} \in \mathcal{Y}^{\phi}(\boldsymbol{r}_{\phi}), \forall \boldsymbol{r}_{\phi}, \forall \phi \in \Phi, \tag{19c}$$

$$\boldsymbol{C}^{\phi}(\boldsymbol{r}_{\phi}, \boldsymbol{y}^{\phi}) \in \mathcal{C}^{\phi}(\boldsymbol{r}_{\phi}, \boldsymbol{y}^{\phi}), \forall \boldsymbol{y}_{\phi} \in \mathcal{Y}^{\phi}(\boldsymbol{r}_{\phi}), \forall \boldsymbol{r}_{\phi}, \forall \phi \in \Phi, \tag{19d}$$

$$s_{\psi} \in \{0,1\}, \forall \psi \in \Psi. \tag{19e}$$

Here, we define an integer decision vector $\boldsymbol{s} = \{s_{\psi} \in \{0,1\}, \forall \psi \in \Psi\}$, where $s_{\psi} = 1$ indicates that the scenario ψ is selected; otherwise, $s_{\psi} = 0$. In addition, $\boldsymbol{y}^{\phi}$ denotes the decision variable $\boldsymbol{y}$ under scenario ϕ, and $\boldsymbol{C}^{\phi}$ represents the completion time vector for all tasks currently assigned to the workstation associated with ϕ. In alignment with the

function $G(\cdot)$ defined in (14) of the ADP model, the objective function is defined as:

$$g^{\phi}\left(\boldsymbol{r}_{\phi}, \boldsymbol{y}^{\phi}, \boldsymbol{C}^{\phi}\right) = \frac{1-\gamma}{|\mathcal{J}|} C^{\phi}_{\max} + \frac{\gamma}{|\mathcal{K}|} \sum_{i_{\phi} \in \mathcal{I}^{\phi}} C^{\phi}_{i_{\phi}} - (\frac{1-\gamma}{|\mathcal{J}|} + \frac{\gamma}{|\mathcal{K}|}) p^{\phi}_{i},$$

where $C^{\phi}_{\max}$ represents the workstation makespan under scenario ϕ, i_{ϕ} is the task index under scenario ϕ, $C^{\phi}_{i_{\phi}}$ is the completion time of task i_{ϕ}, and p^{ϕ}_{i} is the processing time. Constraint (19b) ensures exactly one scenario is selected. Constraint (19c) represents a set of constraints related to the vector $\boldsymbol{y}^{\phi}$, and is transformed from the constraints (1g) $\sim$ (1j), (1n), and (1o) in Model 1. Similarly, Constraint (19d) imposes additional constraints on the vector $\boldsymbol{C}^{\phi}$, and is transformed from the constraints (1b) and (1m) in Model 1. The detailed formulations of constraints (19c) and (19d) are provided in Appendix A.15.

To ensure computational tractability, we define $\boldsymbol{z} = \left\{ z_{\phi}, \forall \phi \in \Phi \,\middle|\, z_{\phi} = \sum_{\psi \in \Psi} a_{\phi,\psi} s_{\psi} \right\}$ as a new decision vector, where $a_{\phi,\psi} = 1$ if $\phi \in \Phi_{\psi}$, otherwise, $a_{\phi,\psi} = 0$. Through the mathematical transformations detailed in Appendix A.13, we can reformulate the scenario-based distributionally robust counterpart (19) into the following type-based version:

$$\min_{\boldsymbol{z}} \sum_{\phi} \left\{ \sup_{f_{\phi} \in \mathscr{F}_{\phi}(\sigma_1, \ldots, \sigma_N)} \mathbb{E}^{\boldsymbol{r}_{\phi}} \left\{ \max_{\boldsymbol{y}^{\phi}} \min_{\boldsymbol{C}^{\phi}} g^{\phi}\left(\boldsymbol{r}_{\phi}, \boldsymbol{y}^{\phi}, \boldsymbol{C}^{\phi}\right) \right\} \right\} z_{\phi} \tag{20a}$$

$$s.t. \quad (19c), (19d),$$

$$\boldsymbol{z} \in \mathcal{Z}, \tag{20b}$$

where $\mathcal{Z}$ defines the feasible region of z_{ϕ}, and can be further specified as:

$$\mathcal{Z} = \left\{ z_{\phi} \in \{0,1\}, \forall \phi \in \Phi \,\middle|\, \sum_{\phi \in \langle i, \cdot, \cdot \rangle} z_{\phi} \leq 1, \forall i \in \mathcal{I}_a, \sum_{\phi \in \langle \cdot, j, \cdot \rangle} z_{\phi} \leq 1, \forall j \in \mathcal{J}_a, \sum_{\phi \in \langle \cdot, \cdot, k \rangle} z_{\phi} = 1, \forall k \in \mathcal{K}_a \right\},$$

where $\mathcal{I}_a$, $\mathcal{J}_a$, and $\mathcal{K}_a$ represent the set of unassigned tasks, idle workstations, and idle AMRs at each decision epoch.

To enhance solving efficiency, we finally decompose Model 20 into a two-stage model, eliminating redundant calculations for identical target types across different scenarios. The main problem in the first stage is a three-dimensional assignment problem as follows:

$$\min_{\boldsymbol{z}} \sum_{\phi} \hat{g}_{\phi} z_{\phi} \tag{21a}$$

$$s.t. \quad \boldsymbol{z} \in \mathcal{Z}. \tag{21b}$$

And the type-based sub-problem in the second stage is given by:

$$\hat{g}_\phi = \left\{ \sup_{f_\phi \in \mathscr{F}^\phi(\sigma_1,\ldots,\sigma_N)} \mathbb{E}^{\boldsymbol{r}_\phi} \left\{ \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \right\} \middle| \text{(19c), (19d)} \right\} \tag{22}$$

The second-stage problem can be further simplified via the following proposition for $\hat{g}_\phi$.

Proposition 5.1

$$\begin{aligned} \hat{g}_\phi &= \sup_{f_\phi \in \mathscr{F}^\phi(\sigma_1,\ldots,\sigma_N)} \int \max_{\boldsymbol{y}^\phi \in \mathcal{Y}(\boldsymbol{r}_\phi)} \min_{\boldsymbol{C}^\phi \in \mathcal{C}(\boldsymbol{r}_\phi, \boldsymbol{y})} g(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi) df(\boldsymbol{r}_\phi) \\ &= \sum_{n=1}^{N} (\sigma_n - \sigma_{n-1}) \max_{\boldsymbol{r}_\phi^n \in \mathscr{R}^n, \boldsymbol{y}^\phi \in \mathcal{Y}(\boldsymbol{r}_\phi^n)} \min_{\boldsymbol{C}^\phi \in \mathcal{C}(\boldsymbol{r}_\phi^n, \boldsymbol{y}^\phi)} g(\boldsymbol{r}_\phi^n, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi). \end{aligned} \tag{23}$$

In addition, Constraint (19d) is a binding equality constraint that uniquely determines the value of $\boldsymbol{C}^\phi$. Hence, the innermost minimization problem $\min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi^n, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right)$ reduces to a constant over the (singleton) feasible region, yielding:

$$g^\phi \left(\boldsymbol{r}_\phi^n, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) = \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi^n, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) = \max_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi^n, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right).$$

Thus, we can simplify the second-stage sub-problem of model 22 from a max-min problem to the following maximization problem:

$$\hat{g}_\phi = \left\{ \sum_{n=1}^{N} (\sigma_n - \sigma_{n-1}) \max_{\boldsymbol{r}_\phi^n \in \mathscr{R}_\phi^n} g^\phi \left(\boldsymbol{r}_\phi^n, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \middle| \text{(19c), (19d)} \right\} \tag{24}$$

Therefore, we reformulate the distributionally robust counterpart (19) into a solvable two-stage mixed integer linear programming (MILP) problem. At each decision epoch, we first solve all feasible second-stage subproblems (24) for feasible types ϕ in parallel, obtaining the precomputed values $\hat{g}_\phi$. These $\hat{g}_\phi$ values are then embedded into the first-stage assignment problem (21). This sequential decomposition exploits the separability of type-level subproblems while maintaining global optimality through the assignment vector $\boldsymbol{z}$.

5.3 *Approximated Optimal Solution for the DRO Sub-problem*

Due to the exponentially growing feasible solutions induced by binary decision variables, the second-stage MILP model remains $\mathcal{NP}$-hard. This complexity poses significant challenges for developing efficient online algorithms capable of meeting real-time scheduling requirements. Therefore, in this subsection, we propose an approximate online allocation

algorithm by constructing a suitable ambiguity set. Therefore, in this subsection, we propose a polynomial-time approximate online allocation algorithm by constructing a tailored ambiguity set. This ambiguity set enables computation of solutions with bounded suboptimality in polynomial time. The formal definition and theoretical guarantees of this ambiguity set are given below. For notational conciseness, we omit the subscripts and superscripts of ϕ and n hereafter.

The ambiguity set under a given confidence level σ_n and type $\phi = \langle i, j, k \rangle$ is given by:

$$\mathscr{R} = \{\boldsymbol{r} | r_i \in [\underline{r}_i, \bar{r}_i], \forall i \in \mathcal{I}, f_l(\boldsymbol{r}) \leq 0, \forall l \in \mathcal{L}\}. \tag{25}$$

Here, $f_l(\boldsymbol{r}) \leq 0,\ \forall l \in \mathcal{L}$ represents a series of constraints about the ambiguity set, satisfying $\frac{\partial f_l(\boldsymbol{r})}{\partial r_i} > 0,\ \frac{\partial f_l(r)}{\partial r_i'} \leq 0,\ \forall i' \neq i$. These conditions stipulate that if the release time of one task increases, the ambiguity set of release times for other tasks does not shrink. In practical scenarios, when the travel time for one task at the workstation increases, it implies that the current traffic situation is unfavorable, and thus the travel time for other tasks is also likely to increase. Therefore, these conditions align with real-world transportation scenarios.

Moreover, with the above settings, there exists a point $\tilde{\boldsymbol{r}} = (\tilde{r}_1, \ldots, \tilde{r}_{|\mathcal{I}|})$, which satisfies $r_i \leq \tilde{r}_i, \forall i \in \mathcal{I}$ for any point within the feasible region of $\boldsymbol{r} = (r_1, \ldots, r_{|\mathcal{I}|})$. In other words, for any r_i, the corresponding value $\tilde{r}_i$ is the maximum value within the vector $\tilde{\boldsymbol{r}}$, and can be considered an approximate solution. Proposition 5.2 provides a theoretical guarantee in comparison with the optimal solution under the FCFS rule.

Proposition 5.2 *Let* $\boldsymbol{r}^* = \arg\max_{\boldsymbol{r} \in \mathscr{R}} g(\boldsymbol{r}, \boldsymbol{y}, \boldsymbol{C})$*, then we have:*

$$|g(\boldsymbol{r}^*, \boldsymbol{y}, \boldsymbol{C}) - g(\tilde{\boldsymbol{r}}, \boldsymbol{y}, \boldsymbol{C})| \leq \frac{\gamma}{|\mathcal{K}|} \lceil \frac{|\mathcal{I}|}{2} \rceil \lfloor \frac{|\mathcal{I}|}{2} \rfloor (\bar{p} - \underline{p}).$$

Notice that, as the subscripts and superscripts of ϕ and n are omitted, the set of tasks in Proposition 5.2 is essentially $\mathcal{I}_\phi$, which only includes the total tasks allocated to the current workstation (and $|\mathcal{I}_\phi| \leq |\mathcal{K}_j|$ holds inherently). Critically, the maximum gap between the approximated solution and optimal solution under the FCFS rule is no greater than the system loss generated by the FCFS rule. In addition, both the approximated solution and the system optimum (without the FCFS rule) yield objective values no greater than that of the FCFS rule. Consequently, the maximum absolute gap between the approximated solution and system optimum remains bounded by $\frac{\gamma}{|\mathcal{K}|} \lceil \frac{|\mathcal{I}_\phi|}{2} \rceil \lfloor \frac{|\mathcal{I}_\phi|}{2} \rfloor (\bar{p} - \underline{p})$.

Given that the release (arrival) times of all tasks are fixed according to the approximate solution $\tilde{\boldsymbol{r}}$ and the application of the FCFS rule based on these predetermined arrival times, the objective value for solution $\tilde{\boldsymbol{r}}$ is computable in polynomial time. As a result, in subsequent numerical experiments, we incorporate $\tilde{\boldsymbol{r}}$ into the solution of the two-stage DRO framework to enhance the problem-solving efficiency and facilitate online dispatching.

6 Numerical Experiments and Results

Finally, we validate our online allocation strategy using actual data and the commercial simulation platform provided by Geek+ (a leading AMR company based in Beijing, China). The results demonstrate the superiority of our algorithm over the classic greedy algorithm − the traditional practical alternative necessitated by the NP-hard nature of the original optimization problem and the uncertainties of travel times.

6.1 *Data-Driven Ambiguity Sets via Quantile Regression*

We utilize a real dataset from a Geek+ smart warehouse, which comprises data from 1,897 transport tasks performed by robots. Each transport task, corresponding to various transport processes, can be divided into three sub-tasks: pick-up, delivery, and return. A specific sample includes the AMR index, task index, sub-task type, as well as the path and transport distance under both actual and optimal scenarios. In practice, the AMR is programmed to follow the optimal route as closely as possible, automatically resorting to alternative routes when the optimal choice is unavailable, such as when it is occupied by other AMRs. For page limit, the descriptive statistics are given in Appendix B.

Based on the statistical findings presented in Appendix B, the distribution of actual travel times demonstrates considerable instability, exhibiting strong correlations with both travel distances and motion trajectory features (such as the number of turns). In addition, our statistical analysis reveals distinct transport profiles across sub-task types: delivery and return sub-tasks exhibit comparable duration distributions and variability, while pick-up sub-tasks demonstrate significantly shorter and more stable travel times. This divergence arises from fundamental differences in load states − delivery and return sub-tasks require AMRs to operate in a loaded state (carrying pods), while pick-up sub-tasks are performed

in an unloaded state. Moreover, the unloaded state enables AMRs to navigate underneath shelves, bypassing congestion and substantially reducing travel delays. Consequently, we utilize four features derived from optimal routing plans for prediction − specifically, travel distance and number of turns under both loaded and unloaded states − since real-time route information remains unavailable during allocation decision epochs.

Given these complex distributional characteristics, we employ quantile regression rather than point estimation to construct the nested ambiguity sets. This approach provides two key advantages: (1) it captures the travel time distribution more comprehensively, including its tails and overall shape; and (2) it is inherently robust to outliers, while its structure aligns well with the requirements of our nested ambiguity set framework. Consequently, we construct quantile regression models for AMR travel time under three scenarios: unloaded transport, loaded transport, and mixed transport (combining both states). Specifically, we fit a series of linear quantile regression models for the 25th, 50th, and 75th percentiles. Numerical results, as shown in Table 4, Appendix B, confirm that both distance metrics and turn metrics significantly impact transport durations across all scenarios.

6.2 *Polynomial Algorithm and Experiment Settings*

The operational workflow of the proposed DRO framework (see Figure 2) comprises separate offline and online phases. In the offline phase, we construct the warehouse layout and train quantile regression models using historical data. During each decision epoch in the online phase, we identify the current positions of available AMRs and target pods, then use quantile regression outputs to construct ambiguity sets. These ambiguity sets inform the formulation and solution of the second-stage DRO subproblem, whose results guide the first-stage matching process to determine optimal dispatch assignments. To evaluate the performance of our DRO algorithm, we benchmark it against the classic greedy algorithm, which assigns each idle AMR to the task with the shortest predicted travel time.

It is noteworthy that the first-stage problem constitutes a three-dimensional matching problem, which is NP-hard. However, within Geek+'s operational environment, task-workstation assignments are predetermined through system configuration, thereby reducing the problem to a two-dimensional bipartite matching that can be solved in polynomial time. To further improve decision quality, we implement a temporal filtering mechanism: at each

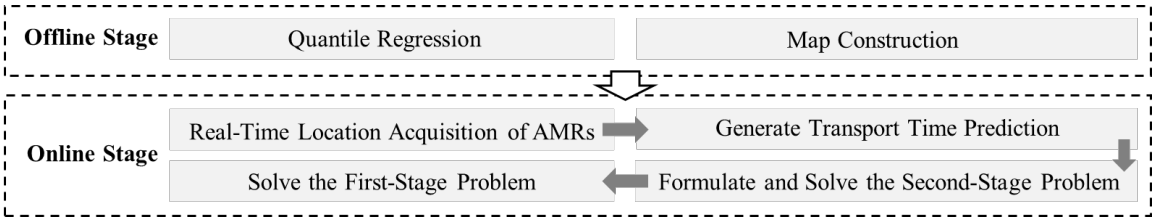


Figure 2: The workflow of the DRO framework

decision epoch, only tasks with predicted travel times below the 75th percentile are considered for immediate allocation. This approach strategically excludes high-duration tasks (those in the upper quartile), as their extended execution windows typically fall outside the current decision epoch's relevance horizon. These deferred tasks are reconsidered in subsequent epochs when updated predictions place them within actionable timeframes.

Our experiments were conducted on Geek+'s commercial simulation platform using a realistic warehouse layout. The setup featured 9 workstations managing a total of 45 AMRs, with each workstation capable of controlling up to 8 AMRs simultaneously. In the simulation environment, we assigned a variety of transport tasks to the workstations, with each task defined by a distinct processing time (see Table 5, Appendix B). To ensure robust and fair comparisons, the simulation environment was held constant across all experimental scenariosincluding the initial positions of AMRs, the assignment of pods to workstations, and the spatial locations of all pods.

6.3 *Numerical Results*

In line with Lamballais et al. (2017), we adopt workstation occupancy rate as the evaluation metric, reflecting our focus on scenarios with a limited number of workstations. In fact, maximizing workstation occupancy rates in our scenario is equivalent to minimizing the makespan, since both the set of tasks and their estimated processing times are exogenous. As shown in Table 1, our DRO algorithm consistently outperforms the greedy algorithm across all workstations. On average, the DRO approach achieves an absolute improvement of 9.81% and a relative improvement of 14.87% in workstation occupancy rate.

To isolate the impact of travel time uncertainty, we conducted a counterfactual experiment assuming deterministic travel times perfectly predicted by regression. As shown in Table 6 (Appendix B), deterministic conditions improve average occupancy to 83.27% −

Table 1: Comparison of Algorithms Based on Workstation Occupancy Rate

Number of Workstations	Greedy Algorithm	DRO Algorithm	Absolute Improvement	Relative Improvement
1	66.53%	76.23%	9.70%	14.58%
2	64.17%	74.40%	10.23%	15.94%
3	62.65%	73.23%	10.58%	16.89%
4	64.59%	75.34%	10.75%	16.64%
5	65.84%	77.62%	11.78%	17.89%
6	62.00%	71.20%	9.20%	14.84%
7	73.37%	82.24%	8.87%	12.09%
8	67.89%	78.22%	10.33%	15.22%
9	66.71%	73.56%	6.85%	10.27%
Average	65.97%	75.78%	9.81%	14.87%

7.49% higher than under uncertainty. Crucially, while uncertainty reduces performance for both algorithms, the relative advantage of the DRO algorithm over greedy algorithm decreases from 14.87% to 11.12% under deterministic conditions. This confirms that our DRO algorithm's superiority is particularly pronounced under travel time uncertainty, where it better manages distributional ambiguity.

7 Conclusion

In summary, this study addresses the online joint order assignment and task allocation problem in RMFS. Leveraging Lagrange relaxation for objective decomposition, we propose an ADP framework incorporating a myopic strategy. Simultaneously, to address uncertainties in travel times, we introduce a nested series of ambiguity sets. This transforms the dynamic programming model into a sequence of solvable two-stage DRO models at each decision epoch. To ensure computational tractability, we employ quantile regression on the operational dataset to construct appropriate ambiguity sets, and design a polynomial-time approximation algorithm with a proven approximation guarantee. Numerical experiments using real-world data from Geek+ demonstrate that our approach significantly outperforms the classic greedy algorithm practically.

While this research primarily addresses allocation strategies within RMFS, the proposed framework naturally extends to other practical scenarios that require robust management of

travel time uncertainty. For example, the methodology adapts readily to last-mile delivery, where distribution center processing times parallel workstation dynamics in our model. It is also well suited to low-altitude economy operations, such as drone delivery, where drones dispatched from centralized depots encounter flight duration uncertaintiesclosely analogous to RMFS robot travel time uncertainty. Given these structural similarities, our core algorithm and ambiguity set approach require minimal modification. Effectively managing uncertainty in these domains, however, depends on capturing application-specific risk factors: (1) In last-mile delivery, uncertainty extends beyond standard transit metrics to operational dynamics, including unpredictable sequencing due to real-time batching, stochastic service times at pickup/drop-off points (e.g., parking delays, customer interactions), and systemic volatility from traffic incidents or urban mobility constraints. (2) In drone delivery, while transit uncertainty remains relevant, dominant factors shift to weather disruptions, airspace access limitations, and stochastic battery performance. Therefore, successful implementation relies less on altering the underlying DRO framework and more on refining uncertainty characterization − mapping domain-specific risks to quantifiable travel time distributions. Advanced forecasting or physics-informed modeling can incorporate these factors into ambiguity sets while preserving computational tractability. Despite these contextual challenges, our study establishes a foundational DRO framework for real-time logistics optimization, providing essential insights into unified ambiguity set construction, efficient approximation algorithms, and adaptive decision-making − enabling deployment across automated warehouses, last-mile deliveries, and emerging aerial fulfillment systems.

Data Availability Statement

The data, codes and numerical results that support the findings of this study are openly available at http://doi.org/10.5281/zenodo.16011215.

References

Averbakh, I. (2010). On-line integrated production–distribution scheduling problems with capacitated deliveries. *European Journal of Operational Research*, 200(2):377–384.

Boysen, N., Briskorn, D., and Emde, S. (2017). Parts-to-picker based order processing in a rack-moving mobile robots environment. *European Journal of Operational Research*, 262(2):550–562.

Carlsson, J. G., Behroozi, M., and Mihic, K. (2018). Wasserstein distance and the distributionally robust tsp. *Operations Research*, 66(6):1603–1624.

Carlsson, J. G. and Delage, E. (2013). Robust partitioning for stochastic multivehicle routing. *Operations research*, 61(3):727–744.

Chen, B. and Vestjens, A. P. (1997). Scheduling on identical machines: How good is lpt in an on-line setting? *Operations research letters*, 21(4):165–169.

Chen, Z.-L. and Hall, N. G. (2022). *Supply Chain Scheduling*. Springer Cham.

Cheng, T. and Kahlbacher, H. (1993). Scheduling with delivery and earliness penalties. *Asia-Pacific Journal of Operational Research*, 10(2):145–152.

da Costa Barros, Í. R. and Nascimento, T. P. (2021). Robotic mobile fulfillment systems: A survey on recent developments and research opportunities. *Robotics and Autonomous Systems*, 137:103729.

Delage, E. and Mannor, S. (2010). Percentile optimization for markov decision processes with parameter uncertainty. *Operations research*, 58(1):203–213.

Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM journal on Applied Mathematics*, 17(2):416–429.

Hoogeveen, J. and Vestjens, A. P. (2000). A best possible deterministic on-line algorithm for minimizing maximum delivery time on a single machine. *SIAM Journal on Discrete Mathematics*, 13(1):56–63.

Kim, H.-J., Pais, C., and Shen, Z.-J. M. (2020). Item assignment problem in a robotic mobile fulfillment system. *IEEE Transactions on Automation Science and Engineering*, 17(4):1854–1867.

Lamballais, T., Merschformann, M., Roy, D., de Koster, M., Azadeh, K., and Suhl, L. (2022). Dynamic policies for resource reallocation in a robotic mobile fulfillment system with time-varying demand. *European Journal of Operational Research*, 300(3):937–952.

Lamballais, T., Roy, D., and De Koster, M. (2017). Estimating performance in a robotic mobile fulfillment system. *European Journal of Operational Research*, 256(3):976–990.

Lamballais Tessensohn, T., Roy, D., and De Koster, R. B. (2020). Inventory allocation in robotic mobile fulfillment systems. *IISE transactions*, 52(1):1–17.

Lee, C.-Y. and Chen, Z.-L. (2001). Machine scheduling with transportation considerations. *Journal of scheduling*, 4(1):3–24.

Liu, S., He, L., and Shen, Z. (2020). On-time last-mile delivery: Order assignment with travel-time predictors. *Management Science*.

Merschformann, M., Lamballais, T., De Koster, M., and Suhl, L. (2019). Decision rules for robotic mobile fulfillment systems. *Operations Research Perspectives*, 6:100128.

Michael, P. (1995). Scheduling. theory, algorithms and systems. *ISBN0-13-706757-7*.

Potts, C. N. (1980). Analysis of a heuristic for one machine sequencing with release dates and delivery times. *Operations Research*, 28(6):1436–1441.

Pundoor, G. and Chen, Z.-L. (2005). Scheduling a production–distribution system to optimize the tradeoff between delivery tardiness and distribution cost. *Naval Research Logistics (NRL)*, 52(6):571–589.

Shah, B. and Khanzode, V. (2017). A comprehensive review of warehouse operational issues. *International Journal of Logistics Systems and Management*, 26(3):346–378.

Tang, L., Li, F., and Chen, Z.-L. (2019). Integrated scheduling of production and two-stage delivery of make-to-order products: Offline and online algorithms. *INFORMS Journal on Computing*, 31(3):493–514.

Tian, J., Fu, R., and Yuan, J. (2007). On-line scheduling with delivery time on a single batch machine. *Theoretical Computer Science*, 374(1-3):49–57.

Ulmer, M. W., Thomas, B. W., Campbell, A., and Woyak, N. (2021). The restaurant meal delivery problem: Dynamic pickup and delivery with deadlines and random ready times. *Transportation Science*, 55:75–100.

Valle, C. A. and Beasley, J. E. (2021). Order allocation, rack allocation and rack sequencing for pickers in a mobile rack environment. *Computers & Operations Research*, 125:105090.

Wang, Z., Sheu, J.-B., Teo, C.-P., and Xue, G. (2022). Robot scheduling for mobile-rack warehouses: Human–robot coordinated order picking systems. *Production and Operations Management*, 31(1):98–116.

Wiesemann, W., Kuhn, D., and Sim, M. (2014). Distributionally robust convex optimization. *Operations research*, 62(6):1358–1376.

Woeginger, G. J. (1994). Heuristics for parallel machine scheduling with delivery times. *Acta Informatica*, 31:503–512.

Xu, H. and Mannor, S. (2012). Distributionally robust markov decision processes. *Mathematics of Operations Research*, 37(2):288–300.

Yang, P., Zhao, Z., and Shen, Z.-J. M. (2021). A flow picking system for order fulfillment in e-commerce warehouses. *IISE Transactions*, 53(5):541–551.

Zhen, L., Tan, Z., de Koster, R., and Wang, S. (2023). How to deploy robotic mobile fulfillment systems. *Transportation Science*, 57(6):1671–1695.

Appendices

A Mathematical Proofs and Transformation Details

A.1 Proof of Proposition 3.1

1. **- Proof of Proposition 3.1-(i):** Let i^* denote a task that has the maximum sum of travel time τ_{0i^*} from the origin 0 and processing time p_{i^*}, i.e., $i^* \in_i \{\tau_{0i} + p_i\}$. Then according to Assumptions 3.1 and 3.2, there is no scenario where the completion time of task i^* is earlier than $\tau_{0i^*} + p_{i^*}$. Therefore, the optimal solution will not be less than $\tau_{0i^*} + p_{i^*}$, providing a lower bound on the optimal solution to the problem. Meanwhile, for $|\mathcal{I}| \leq \min\{|\mathcal{J}|, |\mathcal{K}|\}$, we can obtain a feasible solution with a value equal to the lower bound $\tau_{0i^*} + p_{i^*}$ by assigning each task a dedicated workstation and a dedicated AMR. Therefore, when $|\mathcal{I}| \leq \min\{|\mathcal{J}|, |\mathcal{K}|\}$, this feasible solution is an optimal solution for the problem. Finally, this problem is equivalent to finding the largest element in a set, which can be solved in polynomial time. □

2. **- Proof of Proposition 3.1-(ii):** We first introduce the following lemma before proving Proposition 3.1-2.

   Lemma A.1 *If $|\mathcal{K}| \leq \min\{|\mathcal{I}|, |\mathcal{J}|\}$, then an optimal solution exists such that all tasks on the AMRs are served by their corresponding dedicated workstations.*

   Proof of Lemma A.1:

   We prove Lemma A.1 by demonstrating that for any feasible solution, we can derive a new feasible solution where all tasks on the AMR are served by their dedicated workstation, and the objective function of this new feasible solution is not greater than that of the original solution.

   Specifically, given any feasible solution, when a task is transferred from an AMR to a workstation, the task must wait in the queue if there are other tasks at the workstation that need to be processed first. However, when we pair each AMR with a dedicated workstation, the task no longer needs to wait upon arrival at the workstation and can begin processing immediately. Therefore, for each feasible solution, we preserve the

order of all AMRs on the task and assign each AMR a corresponding workstation, resulting in a new feasible solution. According to Assumption 3.1, the completion time of all tasks in the new feasible solution is not greater than the completion time corresponding to the original feasible solution, and the objective function of the new feasible solution is not greater than that of the original solution. □

According to Lemma A.1, we can consider each AMR and its corresponding workstation as a parallel machine. We define the time from task i leaving the workstation to task i' arriving at the workstation as the sequence-dependent setup time $s_{ii'} = \tau_{ii'}$. And the problem is equivalent to $P_{|\mathcal{K}|}|s_{ii'}|C_{\max}$. It is well known that two special cases of $P_{|\mathcal{K}|}|s_{ii'}|C_{\max}$, namely, $P_{|\mathcal{K}|}||C_{\max}, |\mathcal{K}| \geq 2$ and $|s_{ii'}|C_{\max}$ are both $\mathcal{NP}$-hard problems. Thus, $P_{|\mathcal{K}|}|s_{ii'}|C_{\max}$ is a $\mathcal{NP}$-hard problem. □

3. **- Proof of Proposition 3.1-(iii):** According to Assumptions 3.1 and 3.2, there is no scenario where the arrival time of task i at any given workstation is earlier than τ_{0i}. When $|\mathcal{I}| \leq |\mathcal{K}|$, it is optimal for each task to be transported by a dedicated AMR, i.e., each AMR transports no more than one task. Therefore, we define the release time of each task i as $r_i = \tau_{0i}$, and the problem reduces to $P_{|\mathcal{J}|}|r_i|C_{\max}$ problem.

   It is well known that $P_2||C_{\max}$, a special case of $P_{|\mathcal{J}|}|r_i|C_{\max}$, is $\mathcal{NP}$-hard as it is equivalent to the PARTITION problem. Additionally, Chen and Vestjens (1997) have shown that the longest processing time (LPT) rule is a 3/2-approximation algorithm for $P_{|\mathcal{J}|}|r_i|C_{\max}$. □

4. **- Proof of Proposition 3.1-(iv):** The problem reduces to $P_{|\mathcal{K}|}|s_{ii'}|C_{\max}$ when $p_i = 0, \forall i \in \mathcal{I}$, which is a $\mathcal{NP}$-hard problem. □

A.2 Linearization of Constraints in Model 1

Constraint (1k) can be linearized as:

$$C_i + \tau_{ii'} - M(1 - x_{ii'}^k) \leq u_{i'}, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall k \in \mathcal{K},$$
$$C_i + \tau_{ii'} + M(1 - x_{ii'}^k) \geq u_{i'}, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall k \in \mathcal{K}.$$

Constraint (1l) can be linearized as:

$$C_i - M(1 - y_{ii'}^j) \leq v_{i'}, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall j \in \mathcal{J},$$
$$C_i + M(1 - y_{ii'}^j) \geq v_{i'}, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall j \in \mathcal{J}.$$

Constraint (1m) can be linearized as:

$$u_i + p_i + M(1 - \delta_i) \geq C_i, \forall i \in \mathcal{I},$$
$$v_i + p_i + M\delta_i \geq C_i, \forall i \in \mathcal{I},$$
$$u_i - M\delta_i \leq v_i, \forall i \in \mathcal{I},$$
$$u_i + p_i \leq C_i, v_i + p_i \leq C_i, \forall i \in \mathcal{I}.$$

Constraint (1n) can be linearized as:

$$u_i - M(1 - y_{ii'}^j) \leq u_{i'}, \forall i \in \mathcal{I}, \forall i' \in \mathcal{I} \backslash \{i\}, \forall j \in \mathcal{J}.$$

A.3 Proof of Proposition 4.1

First, we demonstrate that for any feasible solution, it's possible to derive a new feasible solution that adheres to the condition proposed in Proposition 4.1. Moreover, the objective function value of this new feasible solution does not exceed that of the original feasible solution.

Consider any feasible solution. For each workstation, if the AMR corresponding to the last task on this workstation has other tasks to serve later, we transfer all subsequent tasks from this AMR to the current workstation for processing. This operation does not alter the arrival time of any task, nor does it increase the waiting time for any task. Consequently, the completion time for all tasks in the new feasible solution does not exceed the completion time in the original feasible solution. Thus, the objective function value for the new feasible solution does not exceed that of the original feasible solution.

Therefore, for an optimal solution, we can apply the above operation to derive a new optimal solution. This new solution satisfies the condition that the last task processed at each workstation is the final task of the corresponding AMR. □

A.4 Proof of Proposition 4.2

First, consider any feasible solution that satisfies the valid inequality (2) but does not meet the condition (3). Suppose there are $m \leq |\mathcal{J}|$ workstations with a completion time $C^j_{\max}$ equal to the total completion time $C_{\max}$. We can perform an operation to transfer one of the m tasks with the longest completion time to the workstation with the shortest processing time, thereby obtaining a new feasible solution. Since the original feasible solution does not satisfy the condition (3), the completion time of the transferred task in the new solution is less than its original completion time. Consequently, the objective function of the new feasible solution is not greater than that of the original feasible solution, and the new feasible solution includes $(m-1) > 0$ workstations with a completion time $C^j_{\max}$ equal to the total completion time $C_{\max}$.

In fact, for any optimal solution satisfying the valid inequality (2), with m workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$, we can obtain a new optimal solution that satisfies the condition (3) by performing the above operation no more than $(m-1)$ times. After performing the operation once, the objective function of the new feasible solution is not greater than the objective function of the original feasible solution. Therefore, performing this operation on an optimal solution that satisfies the valid inequality (2) but does not meet the condition (3) will yield another optimal solution.

We then prove by contradiction that the operation will be performed no more than $(m-1)$ times. Suppose there exist optimal solutions that, even after $(m-1)$ operations, still do not satisfy the condition (3), and the operation can still be continued to obtain a new optimal solution. Since we have proven that each operation reduces by one the number of workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$, after $(m-1)$ operations, only one workstation remains with a completion time equal to $C_{\max}$. If the operation is performed again in this case, the objective function of the new solution obtained is strictly less than that of the original solution. This contradicts the assumption that the original solution is optimal.

Therefore, for any optimal solution that satisfies the valid inequality (2) but does not meet the condition (3), we can perform the above operation no more than $(m-1)$ times to obtain a new optimal solution that satisfies the condition (3). □

A.5 Proof of Proposition 4.3

The proof of Proposition 4.3 follows a similar logic to that of Proposition 4.2. First, consider any feasible solution that does not satisfy either condition (4) or (5), and has $m \leq |\mathcal{J}|$ workstations with a completion time $C^j_{\max}$ equal to the total completion time $C_{\max}$. We can perform an operation to transfer one of the m tasks with the longest completion time to the workstation with the shortest corresponding completion time on the AMR with the shortest corresponding completion time, thereby obtaining a new feasible solution. Since the original feasible solution does not satisfy conditions (4) and (5), the completion time of the transferred task in the new solution is less than its original completion time. Consequently, the objective function of the new feasible solution is not greater than that of the original feasible solution, and the new feasible solution includes $(m-1) > 0$ workstations with a completion time $C^j_{\max}$ equal to the total completion time $C_{\max}$.

Then, for any optimal solution with m workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$, we can obtain a new optimal solution that satisfies at least one of the conditions (4) and (5) by performing the above operation no more than $(m-1)$ times. After performing the operation once, the objective function of the new feasible solution is not greater than the objective function of the original feasible solution. Therefore, performing this operation on an optimal solution that does not satisfy conditions (4) and (5) will yield another optimal solution.

We finally prove by contradiction that the operation will be performed no more than $(m-1)$ times. Suppose there exist optimal solutions that, even after $(m-1)$ operations, still do not satisfy either condition (4) or (5), and the operation can still be continued to obtain a new optimal solution. Since we have proven that each operation reduces by one the number of workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$, after $(m-1)$ operations, only one workstation remains with a completion time equal to $C_{\max}$. If the operation is performed again in this case, the objective function of the new solution obtained is strictly less than that of the original solution. This contradicts the assumption that the original solution is optimal. Therefore, for any optimal solution that does not satisfy either condition (4) or (5), we can perform the above operation no more than $(m-1)$ times to obtain a new optimal solution that satisfies at least one of the two conditions. □

A.6 Proof of Proposition 4.4

We continue to apply the operation proposed in the proof of Proposition 4.2 and prove Proposition 4.4 by contradiction. We assume that there exist optimal solutions that do not satisfy condition (6). We then consider optimal solutions that satisfy the condition where there are $m \leq |\mathcal{J}|$ workstations whose completion time $C^j_{\max}$ is equal to the total completion time $C_{\max}$. Then we have the following two scenarios.

1. When $m = |\mathcal{J}|$, we have $|\mathcal{J}|C_{\max} - \sum_{j\in\mathcal{J}} C^j_{\max} = 0$, which violates the assumption, hence a contradiction is derived.

2. When $m \leq |\mathcal{J}| - 1$, for $|\mathcal{J}|C_{\max} - \sum_{j\in\mathcal{J}} C^j_{\max} > (|\mathcal{J}| - 1)(\bar{\tau} + \bar{p})$, we have the following results. Excluding the workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$, the sum of the completion times of the remaining $(|\mathcal{J}| - m)$ workstations is greater than $(|\mathcal{J}| - 1)(\bar{\tau} + \bar{p})$. Therefore, there is at least one workstation whose completion time $C^j_{\max}$ is less than $(C_{\max} - \bar{\tau} - \bar{p})$. We can then perform the operation proposed in the proof of Proposition 4.2 to obtain a new optimal solution. Each time an operation is performed, the completion time of the selected workstation with the shortest completion time does not increase by more than $\bar{\tau} + \bar{p}$. Therefore, after performing $(m-1)$ operations, the sum of the completion times of the $(|\mathcal{J}| - m)$ workstations is greater than $(|\mathcal{J}| - m)(\bar{\tau} + \bar{p})$. There is still at least one workstation whose completion time $C^j_{\max}$ is less than $(C_{\max} - \bar{\tau} - \bar{p})$, and we can continue the operation to obtain a new optimal solution. This contradicts the previous result proven in Proposition 4.2 that the operation will not exceed $(m - 1)$ times.

Taken together, the optimal solutions satisfy condition (6), and Proposition 4.4 is proved. □

A.7 Proof of Proposition 4.5

We continue to apply the operation proposed in the proof of Proposition 4.3 and prove Proposition 4.5 by contradiction. We consider optimal solutions such that there are $m \leq |\mathcal{J}|$ workstations whose completion time $C^j_{\max}$ is equal to the total completion time $C_{\max}$.

1. When $m = |\mathcal{J}|$, we have $|\mathcal{J}|C_{\max} - \sum_{j\in\mathcal{J}} C^j_{\max} = 0$. Obviously, the assumption that the optimal solution does not satisfy condition is violated, yielding a contradiction.

2. When $m \leq |\mathcal{J}| - 1$, for $|\mathcal{K}|C_{\max} - \sum_{k\in\mathcal{K}} C^k_{\max} > (|\mathcal{K}| - 1)(\bar{\tau} + \bar{p})$ and $|\mathcal{J}|C_{\max} - \sum_{j\in\mathcal{J}} C^j_{\max} > (|\mathcal{J}| - 1)\bar{p}$, we have the following results. Excluding the workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$, the sum of the completion times of the remaining $(|\mathcal{J}| - m)$ workstations is greater than $(|\mathcal{J}| - 1)\bar{p}$. Similarly, excluding the AMRs whose completion time $C^k_{\max}$ equals the total completion time $C_{\max}$, the sum of the completion times of the remaining $(|\mathcal{K}| - m)$ AMRs is greater than $(|\mathcal{K}| - 1)(\bar{\tau} + \bar{p})$. Therefore, there is at least one workstation whose completion time $C^j_{\max}$ is less than $(C_{\max} - \bar{p})$, and there is at least one AMR whose completion time $C^k_{\max}$ is less than $(C_{\max} - \bar{\tau} - \bar{p})$. And we can perform the operation proposed in the proof of Proposition 4.3 and provide a new optimal solution. Every time an operation is performed, the completion time of the selected shortest completion time workstation does not increase by more than $\bar{p}$, and the completion time of the selected shortest completion time AMR does not increase by more than $\bar{\tau} + \bar{p}$. Therefore, after performing $(m - 1)$ operations, the sum of the completion times of the $(|\mathcal{J}| - m)$ workstations is greater than $(|\mathcal{J}| - m)\bar{p}$, and the sum of the completion times of the $(|\mathcal{K}| - m)$ AMRs is greater than $(|\mathcal{K}| - m)(\bar{\tau} + \bar{p})$. There is still at least one workstation whose completion time $C^j_{\max}$ is less than $(C_{\max} - \bar{p})$, and at least one AMR whose completion time $C^k_{\max}$ is less than $(C_{\max} - \bar{\tau} - \bar{p})$. We can continue to apply the operation to obtain a new optimal solution. This contradicts the previous result that the operation will not exceed $(m-1)$ times proved in Proposition 4.3.

In conclusion, the optimal solutions satisfy at least one of the two conditions (7) and (8), and Proposition 4.5 is proved. □

A.8 Corollaries in Section 3

Corollary A.1 *Under the valid inequality* (2)*, the optimal solutions satisfy the condition*

$$C_{\max} - \min_{j\in\mathcal{J}} C^j_{\max} \leq m(\bar{\tau} + \bar{p}). \tag{26}$$

where $m \leq |\mathcal{J}|$ represents the total number of workstations whose completion time $C^j_{\max}$ is equal to the total completion time $C_{\max}$.

Proof of Corollary A.1. We regard the m workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$ and the workstation with the shortest total completion time $C^j_{\max}$ as a subsystem. Since the goal of the subsystem determines the goal of the overall system, the optimality of the subsystem is a necessary condition for the optimality of the overall system. Applying the necessary condition proposed in Proposition 4.4 to the sub-problem, we have

$$(m+1)C_{\max} - mC_{\max} - \min_{j\in\mathcal{J}} C^j_{\max} \leq m(\bar{\tau}+\bar{p}) \quad \Rightarrow \quad C_{\max} - \min_{j\in\mathcal{J}} C^j_{\max} \leq m(\bar{\tau}+\bar{p}). \tag{27}$$

Thus, when the overall system is optimal, the necessary condition must be established and the Corollary is proved. □

Corollary A.2 *Under the valid inequality* (2)*, the optimal solutions satisfy at least one of the following two bounds:*

i. AMR-related bound:

$$C_{\max} - \min_{k\in\mathcal{K}} C^k_{\max} \leq m(\bar{\tau}+\bar{p}), \tag{28}$$

ii. Workstation-related bound:

$$C_{\max} - \min_{j\in\mathcal{J}} C^j_{\max} \leq m\bar{p}, \tag{29}$$

where $m \leq |\mathcal{J}|$ represents the total number of workstations whose completion time $C^j_{\max}$ is equal to the total completion time $C_{\max}$.

Proof of Corollary A.2. We regard the m workstations whose completion time $C^j_{\max}$ equals the total completion time $C_{\max}$ and the workstation with the shortest total completion time $C^j_{\max}$, and the m AMRs whose completion time $C^k_{\max}$ equals the total completion time $C_{\max}$ and the AMR with the shortest total completion time $C^k_{\max}$ as a subsystem. Since the goal of the subsystem determines the goal of the overall system, the optimality of the subsystem is a necessary condition for the optimality of the overall system. Applying the necessary condition proposed in Proposition 4.5 to the sub-problem, at least one of the following two conditions is established,

$$(m+1)C_{\max} - mC_{\max} - \min_{k\in\mathcal{K}} C^k_{\max} \leq m(\bar{\tau}+\bar{p}) \quad \Rightarrow \quad C_{\max} - \min_{k\in\mathcal{K}} C^k_{\max} \leq m(\bar{\tau}+\bar{p}), \tag{30}$$

$$(m+1)C_{\max} - mC_{\max} - \min_{j\in\mathcal{J}} C^j_{\max} \le m\bar{p} \quad \Rightarrow \quad C_{\max} - \min_{j\in\mathcal{J}} C^j_{\max} \le m\bar{p}. \tag{31}$$

Thus, when the overall system is optimal, at least one of the two conditions is established and the Corollary is proved. □

A.9 Mathematical Transformation of the Lagrange Relaxation Problem

The transformation from Eq. (9) to Eq. (10) is given as follows:

$$\begin{aligned}
& C_{\max} - \sum_{k\in\mathcal{K}} \lambda_k (C_{\max} - \sum_{i\in\mathcal{I}} C_i x^k_{i,|\mathcal{I}|+1}) - \sum_{j\in\mathcal{J}} \mu_j (C_{\max} - \sum_{i\in\mathcal{I}} C_i y^j_{i,|\mathcal{I}|+1}) \\
= \quad & (1 - \sum_{k\in\mathcal{K}} \lambda_k - \sum_{j\in\mathcal{J}} \mu_j) C_{\max} + \sum_{k\in\mathcal{K}} \lambda_k \sum_{i\in\mathcal{I}} C_i x^k_{i,|\mathcal{I}|+1} + \sum_{j\in\mathcal{J}} \mu_j \sum_{i\in\mathcal{I}} C_i y^j_{i,|\mathcal{I}|+1} \\
= \quad & \sum_{k\in\mathcal{K}} \lambda_k \sum_{i\in\mathcal{I}} C_i x^k_{i,|\mathcal{I}|+1} + \sum_{j\in\mathcal{J}} \mu_j \sum_{i\in\mathcal{I}} C_i y^j_{i,|\mathcal{I}|+1} \\
= \quad & \sum_{k\in\mathcal{K}} \lambda_k \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}\setminus i} (C_{i'} - C_i) x^k_{ii'} + \sum_{j\in\mathcal{J}} \mu_j \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}\setminus i} (C_{i'} - C_i) y^j_{ii'} \\
= \quad & \sum_{k\in\mathcal{K}} \lambda_k \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} (C_{i'} - C_i) x^k_{ii'} + \sum_{j\in\mathcal{J}} \mu_j \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} (C_{i'} - C_i) y^j_{ii'} \\
= \quad & \sum_{k\in\mathcal{K}} \lambda_k \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} (\max\{u_{i'}, v_{i'}\} + p_{i'} - C_i) x^k_{ii'} + \sum_{j\in\mathcal{J}} \mu_j \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} (\max\{u_{i'}, v_{i'}\} + p_{i'} - C_i) y^j_{ii'} \\
= \quad & \sum_{k\in\mathcal{K}} \lambda_k \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} (\max\{u_{i'}, v_{i'}\} + p_{i'} + \tau_{ii'} - u_{i'}) x^k_{ii'} + \sum_{j\in\mathcal{J}} \mu_j \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} (\max\{u_{i'}, v_{i'}\} + p_{i'} - v_{i'}) y^j_{ii'} \\
= \quad & \sum_{k\in\mathcal{K}} \lambda_k \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} ((v_{i'} - u_{i'})^+ + p_{i'} + \tau_{ii'}) x^k_{ii'} + \sum_{j\in\mathcal{J}} \mu_j \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} ((u_{i'} - v_{i'})^+ + p_{i'}) y^j_{ii'}.
\end{aligned}$$

And the transformation from (10) to (11) is given as follows:

$$\begin{aligned}
& \sum_{k\in\mathcal{K}} \frac{\gamma}{|\mathcal{K}|} \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} ((v_{i'} - u_{i'})^+ + p_{i'} + \tau_{ii'}) x^k_{ii'} + \sum_{j\in\mathcal{J}} \frac{1-\gamma}{|\mathcal{J}|} \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} ((u_{i'} - v_{i'})^+ + p_{i'}) y^j_{ii'} \\
= \quad & \frac{\gamma}{|\mathcal{K}|} \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} ((v_{i'} - u_{i'})^+ + p_{i'} + \tau_{ii'}) \sum_{k\in\mathcal{K}} x^k_{ii'} + \frac{1-\gamma}{|\mathcal{J}|} \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} ((u_{i'} - v_{i'})^+ + p_{i'}) \sum_{j\in\mathcal{J}} y^j_{ii'} \\
= \quad & \frac{\gamma}{|\mathcal{K}|} \sum_{i'\in\mathcal{I}} ((v_{i'} - u_{i'})^+ + p_{i'}) \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{k\in\mathcal{K}} x^k_{ii'} + \frac{\gamma}{|\mathcal{K}|} \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} \tau_{ii'} \sum_{k\in\mathcal{K}} x^k_{ii'} \\
+ \quad & \frac{1-\gamma}{|\mathcal{J}|} \sum_{i'\in\mathcal{I}} ((u_{i'} - v_{i'})^+ + p_{i'}) \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{j\in\mathcal{J}} y^j_{ii'} \\
= \quad & \sum_{i'\in\mathcal{I}} \frac{\gamma}{|\mathcal{K}|} ((v_{i'} - u_{i'})^+ + p_{i'}) + \frac{1-\gamma}{|\mathcal{J}|} ((u_{i'} - v_{i'})^+ + p_{i'}) + \frac{\gamma}{|\mathcal{K}|} \sum_{i\in\mathcal{I}\cup\{0\}} \sum_{i'\in\mathcal{I}} \tau_{ii'} \sum_{k\in\mathcal{K}} x^k_{ii'} \\
= \quad & \sum_{i'\in\mathcal{I}} \frac{\gamma}{|\mathcal{K}|} (v_{i'} - u_{i'})^+ + \frac{1-\gamma}{|\mathcal{J}|} (u_{i'} - v_{i'})^+ + (\frac{\gamma}{|\mathcal{K}|} + \frac{1-\gamma}{|\mathcal{J}|}) p_{i'} + \frac{\gamma}{|\mathcal{K}|} \sum_{i\in\mathcal{I}\cup\{0\}} \tau_{ii'} \sum_{k\in\mathcal{K}} x^k_{ii'}.
\end{aligned}$$

A.10 Proof of Lemma 4.1

To prove Lemma 4.1, we introduce the following lemma.

Lemma A.2 *(Rearrangement Inequality) Let* $0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_K, 0 \leq \phi_1 \leq \phi_2 \leq \cdots \leq \phi_K$*, for any order* $[k]$*,*

$$\lambda_1\phi_K + \lambda_2\phi_{K-1} + \cdots + \lambda_K\phi_1 \leq \lambda_1\phi_{[1]} + \lambda_2\phi_{[2]} + \cdots + \lambda_K\phi_{[K]},$$

and

$$\lambda_1\phi_K + \lambda_2\phi_2 + \cdots + \lambda_K\phi_K \geq \lambda_1\phi_{[1]} + \lambda_2\phi_{[2]} + \cdots + \lambda_K\phi_{[K]}.$$

From lemma A.2, when $\lambda_1 + \lambda_2 + \cdots + \lambda_K = \Lambda$, $0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_K$, $0 \leq \phi_1 \leq \phi_2 \leq \cdots \leq \phi_K$, we have

$$\begin{aligned}
&\lambda_1\phi_K + \lambda_2\phi_{K-1} + \cdots + \lambda_K\phi_1 \leq \lambda_1\phi_1 + \lambda_2\phi_2 + \cdots + \lambda_{K-1}\phi_{K-1} + \lambda_K\phi_K \\
&\lambda_1\phi_K + \lambda_2\phi_{K-1} + \cdots + \lambda_K\phi_1 \leq \lambda_1\phi_2 + \lambda_2\phi_3 + \cdots + \lambda_{K-1}\phi_K + \lambda_K\phi_1 \\
&\lambda_1\phi_K + \lambda_2\phi_{K-1} + \cdots + \lambda_K\phi_1 \leq \lambda_1\phi_3 + \lambda_2\phi_4 + \cdots + \lambda_{K-1}\phi_1 + \lambda_K\phi_2 \\
&\cdots \\
&\lambda_1\phi_K + \lambda_2\phi_{K-1} + \cdots + \lambda_K\phi_1 \leq \lambda_1\phi_K + \lambda_2\phi_1 + \cdots + \lambda_{K-1}\phi_{K-2} + \lambda_K\phi_{K-1}
\end{aligned}$$

Summing up the above inequalities, we obtain

$$\begin{aligned}
n(\lambda_1\phi_K + \lambda_2\phi_{K-1} + \cdots + \lambda_K\phi_1) &\leq \lambda_1 \sum_{k=1}^{K} \phi_k + \lambda_2 \sum_{k=1}^{K} \phi_k + \cdots + \lambda_k \sum_{k=1}^{K} \phi_k \\
&= \Lambda \sum_{k=1}^{K} \phi_k.
\end{aligned}$$

Therefore, when $\lambda_1 = \lambda_2 = \cdots = \lambda_K$, or $\phi_1 = \phi_2 = \cdots = \phi_K$, we have

$$\lambda_1\phi_K + \lambda_2\phi_{K-1} + \cdots + \lambda_K\phi_1 = \frac{\Lambda}{n} \sum_{k=1}^{K} \phi_k,$$

which proves Lemma 4.1. □

A.11 Proof of Proposition 4.6

As discussed in Section 4.2, our problem can be considered as a single-machine scheduling problem with release time r_i, where the objective function is $\frac{\gamma}{|\mathcal{K}|}\sum C_i + \frac{1-\gamma}{|\mathcal{J}|}C_{\max}$. We use this model to prove Proposition 4.6.

1. For a processing sequence under ERD rule, we can consider multiple adjacent tasks without idle time as a task segment. By doing this, we can divide the entire set of tasks at points of idle time and create separate task segments. Within each segment, there is no idle time between tasks, and the release time of each task within a segment is not earlier than the start time of the segment. First, we define the following swap operation: if swapping the order of any two adjacent tasks i and i' within each task segment decreases the total objective value, we implement the swap operation. Repeat this operation until the order satisfies the optimal strategy. Assuming that under the ERD rule, task i is processed before task i', i.e., $r_i \leq r_{i'}$. Let w_i be the start time of task i. We have $r_i \leq w_i$. The gap between the objective function before and after the swap is given by: $2w_i + 2p_i + p_{i'} - (2r_{i'} + 2p_{i'} + p_i) = 2w_i - 2r_{i'} + p_i - p_{i'}$. When $r_{i'}$ takes the minimum value and equals r_i, the gap is maximized. Therefore, when all tasks within each task segment have the same release time, the gap between the ERD rule and the optimal strategy is maximized.

2. In each task segment, we demonstrate the maximum gap as follows. Assume that there are N tasks in an arbitrary segment. Without loss of generality, let $\underline{p} \leq p_1 \leq p_2 \leq \cdots \leq p_N \leq \bar{p}$. When $r_1 = r_2 = \cdots = r_N$, the flowtime of any processing sequence under ERD rule $[n]$ is given by

$$\sum_{i=1}^{N} \sum_{n=1}^{i} p_{[n]} = Np_{[1]} + (N-1)p_{[2]} + \cdots + 2p_{[N-1]} + p_{[N]}. \tag{32}$$

From Lemma A.2, for any processing sequence, we have

$$Np_1 + (N-1)p_2 + \cdots + 2p_{N-1} + p_N \leq Np_{[1]} + (N-1)p_{[2]} + \cdots + 2p_{[N-1]} + p_{[N]}, \tag{33}$$

$$Np_N + (N-1)p_{N-1} + \cdots + 2p_2 + p_1 \geq Np_{[1]} + (N-1)p_{[2]} + \cdots + 2p_{[N-1]} + p_{[N]}. \tag{34}$$

Inequality (33) implies that Shortest Processing Time (SPT) first is optimal for $1||\sum C_i$, and Inequality (34) implies that Longest Processing Time (LPT) first policy provide the worst sequence of $1||\sum C_i$. Therefore, the largest gap between different sequences is

$$\begin{aligned} & (Np_N + (N-1)p_{N-1} + \cdots + 2p_2 + p_1) - (Np_1 + (N-1)p_2 + \cdots + 2p_{N-1} + p_N) \\ = \; & (N-1)(p_N - p_1) + (N-3)(p_{N-1} - p_2) + \left(\lceil \frac{N}{2} \rceil + 1 - \lfloor \frac{N}{2} \rfloor \right) (p_{\lceil \frac{N}{2} \rceil + 1} - p_{\lfloor \frac{N}{2} \rfloor}). \end{aligned}$$

For $\underline{p} \leq p_n \leq \bar{p}, \forall n \leq N$, we take $p_n = \underline{p}$, if $n < \frac{N}{2}$, and $p_n = \bar{p}$, if $n > \frac{N}{2}$. And the largest gap is given by

$$\begin{aligned}
& (N-1)(p_N - p_1) + (N-3)(p_{N-1} - p_2) + \left(\lceil \frac{N}{2} \rceil + 1 - \lfloor \frac{N}{2} \rfloor\right)(p_{\lceil \frac{N}{2} \rceil + 1} - p_{\lfloor \frac{N}{2} \rfloor}) \\
\leq \quad & (N-1)(\bar{p} - \underline{p}) + (N-3)(\bar{p} - \underline{p}) + \left(\lceil \frac{N}{2} \rceil + 1 - \lfloor \frac{N}{2} \rfloor\right)(\bar{p} - \underline{p}) \\
= \quad & \left(\left(N + N - 1 + \cdots + \lceil \frac{N}{2} \rceil + 1\right) - \left(1 + 2 + \cdots + \lfloor \frac{N}{2} \rfloor\right)\right)(\bar{p} - \underline{p}) \\
= \quad & \lceil \frac{N}{2} \rceil \lfloor \frac{N}{2} \rfloor (\bar{p} - \underline{p}).
\end{aligned}$$

3. We have proven the maximum gap for each task segment. Now we will prove that when the number of task segments is 1, i.e., under the ERD rule where no idle time exists, the total gap reaches its maximum value. Assuming there are K segments, where the number of tasks in each segment is denoted as N_k, we have $N_1 + N_2 + \cdots + N_K = |\mathcal{I}_j|$, $n_k \geq 0, \forall k \leq K$, the total gap is

$$\sum_{k=1}^{K} \lceil \frac{N_k}{2} \rceil \lfloor \frac{N_k}{2} \rfloor (\bar{p} - \underline{p}) \leq \lceil \frac{|\mathcal{I}_j|}{2} \rceil (\bar{p} - \underline{p}) \sum_{k=1}^{K} \lfloor \frac{N_k}{2} \rfloor \leq \lceil \frac{|\mathcal{I}_j|}{2} \rceil \lfloor \frac{|\mathcal{I}_j|}{2} \rfloor (\bar{p} - \underline{p}). \tag{35}$$

When $N_k = |\mathcal{I}_j|, n_{k'} = 0, \forall k, k' \leq K, k' \neq k$, we have

$$\sum_{k=1}^{K} \lceil \frac{N_k}{2} \rceil \lfloor \frac{N_k}{2} \rfloor (\bar{p} - \underline{p}) = \lceil \frac{|\mathcal{I}_j|}{2} \rceil \lfloor \frac{|\mathcal{I}_j|}{2} \rfloor (\bar{p} - \underline{p}) \tag{36}$$

□

A.12 Proof of Proposition 4.7

Similar to the proof in Proposition 4.6, we first divide all tasks at idle points and create separate task segments. By applying the same analysis method as in Proposition 4.6, we can prove that when all tasks within each task segment have the same release time, the gap between the FCFS sequence and the optimal sequence is maximized under the buffer setting. In addition, similar to Proposition 4.6, when there is only one segment (all tasks have the same release time) under the buffer setting, the gap is maximized.

Thus, we demonstrate the maximum gap under the case when there is only one segment:

1. When the buffer size $|\bar{\mathcal{K}}_j| \geq \lfloor \frac{|\mathcal{I}_j|}{2} \rfloor$, based on part (b) of the proof in Proposition 4.6, we construct a task release order where the first $\lfloor \frac{|\mathcal{I}_j|}{2} \rfloor$ tasks have a processing time of

$\bar{p}$, and all subsequent tasks have a processing time of $\underline{p}$. According to the definition, the buffer problem serves the demands in the buffer first. The reordering problem serves the demands with shorter processing times first. Therefore, the gap between the two sequences is $\lceil \frac{|\mathcal{I}_j|}{2} \rceil \lfloor \frac{|\mathcal{I}_j|}{2} \rfloor (\bar{p} - \underline{p})$.

2. When the buffer size $|\bar{\mathcal{K}}_j| \leq \lfloor \frac{|\mathcal{I}_j|}{2} \rfloor - 1$, since the buffer problem serves the demands in the buffer first, the worst-case scenario is to set the processing time of the demand first loaded into the buffer to $\bar{p}$. On the other hand, the reordering problem serves the demands with shorter processing time first, so the maximum gap between the two approaches is $|\bar{\mathcal{K}}_j|(|\mathcal{I}_j| - |\bar{\mathcal{K}}_j|)(\bar{p} - \underline{p})$.

□

A.13 Mathematical Transformation of the DRO Problem

Based on the linear property of the expectation and our definition of the ambiguity sets of marginal distributions, we can derive the following transformations:

$$
\begin{aligned}
& \min_{\boldsymbol{s}} \sup_{f^{\Psi} \in \mathscr{F}^{\Psi}(\sigma_1,\ldots,\sigma_N)} \mathbb{E}^{\boldsymbol{r}} \left\{ \sum_{\psi \in \Psi} \sum_{\phi \in \Phi_\psi} \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) s_\psi \right\} \\
= \quad & \min_{\boldsymbol{s}} \sup_{f^{\Psi} \in \mathscr{F}^{\Psi}(\sigma_1,\ldots,\sigma_N)} \sum_{\psi \in \Psi} \sum_{\phi \in \Phi} \mathbb{E}^{\boldsymbol{r}} \left\{ \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \right\} a_{\phi,\psi} s_\psi \\
& \qquad \text{(due to the definition of } a_{\phi,\psi}) \\
= \quad & \min_{\boldsymbol{s}} \sum_{\psi \in \Psi} \sup_{f^{\psi} \in \mathscr{F}^{\psi}(\sigma_1,\ldots,\sigma_N)} \sum_{\phi \in \Phi} \mathbb{E}^{\boldsymbol{r}} \left\{ \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \right\} a_{\phi,\psi} s_\psi \\
& \qquad \text{(due to the parameter independency defined in Eq. (16))} \\
= \quad & \min_{\boldsymbol{s}} \sum_{\psi \in \Psi} \sup_{f^{\psi} \in \mathscr{F}^{\psi}(\sigma_1,\ldots,\sigma_N)} \sum_{\phi \in \Phi} \mathbb{E}^{\boldsymbol{r}_\phi} \left\{ \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \right\} a_{\phi,\psi} s_\psi \\
& \qquad \text{(due to the expectation property of the marginal distribution)} \\
= \quad & \min_{\boldsymbol{s}} \sum_{\psi \in \Psi} \sum_{\phi \in \Phi} \left\{ \sup_{f_\phi \in \mathscr{F}_\phi(\sigma_1,\ldots,\sigma_N)} \mathbb{E}^{\boldsymbol{r}_\phi} \left\{ \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \right\} \right\} a_{\phi,\psi} s_\psi \\
& \qquad \text{(due to the definition of marginal distribution in Eq. (17))} \\
= \quad & \min_{\boldsymbol{s}} \sum_{\phi \in \Phi} \left\{ \sup_{f_\phi \in \mathscr{F}_\phi(\sigma_1,\ldots,\sigma_N)} \mathbb{E}^{\boldsymbol{r}_\phi} \left\{ \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \right\} \right\} \sum_{\psi \in \Psi} a_{\phi,\psi} s_\psi \\
& \qquad \text{(due to linear transformation)} \\
= \quad & \min_{\boldsymbol{z}} \sum_{\phi \in \Phi} \left\{ \sup_{f_\phi \in \mathscr{F}_\phi(\sigma_1,\ldots,\sigma_N)} \mathbb{E}^{\boldsymbol{r}_\phi} \left\{ \max_{\boldsymbol{y}^\phi} \min_{\boldsymbol{C}^\phi} g^\phi \left(\boldsymbol{r}_\phi, \boldsymbol{y}^\phi, \boldsymbol{C}^\phi \right) \right\} \right\} z_\phi
\end{aligned}
$$

(due to the definition of z_ϕ).

A.14 Proof of Proposition 5.1

To streamline the expressions, we omit the super and subscripts ϕ in the proof. Let

$$\begin{aligned}\rho_n &= \int_{\boldsymbol{r}\in\mathscr{R}^n} df(\boldsymbol{r}) \in [\sigma_n, 1], \forall n \in \{1, 2, \ldots, N\}, \quad \rho_0 = 0,\\ g^*(\boldsymbol{r}) &= \max_{\boldsymbol{y}\in\mathcal{Y}(\boldsymbol{r})} \min_{\boldsymbol{C}\in\mathcal{C}(\boldsymbol{r},\boldsymbol{y})} g(\boldsymbol{r}, \boldsymbol{y}, \boldsymbol{C}).\end{aligned}$$

Then we have

$$\begin{aligned}&\sup_{f\in\mathscr{F}(\sigma_1,\ldots,\sigma_N)} \int \max_{\boldsymbol{y}\in\mathcal{Y}(\boldsymbol{r})} \min_{\boldsymbol{C}\in\mathcal{C}(\boldsymbol{r},\boldsymbol{y})} g(\boldsymbol{r}, \boldsymbol{y}, \boldsymbol{C}) df(\boldsymbol{r})\\
=\ &\sup_{f\in\mathscr{F}(\sigma_1,\ldots,\sigma_N)} \sum_{n=1}^{N} \int_{\boldsymbol{r}\in\mathscr{R}^n\setminus\mathscr{R}^{n-1}} g^*(\boldsymbol{r}^n) df(\boldsymbol{r})\\
\le\ &\sup_{f\in\mathscr{F}(\sigma_1,\ldots,\sigma_N)} \sum_{n=1}^{N} \int_{\boldsymbol{r}\in\mathscr{R}^n\setminus\mathscr{R}^{n-1}} df(\boldsymbol{r}) \max_{\boldsymbol{r}\in\mathscr{R}^n} g^*(\boldsymbol{r})\\
=\ &\max_{\rho_n\in[\sigma_n,1],\forall n\le N} \sum_{n=1}^{N} (\rho_n - \rho_{n-1}) \max_{\boldsymbol{r}\in\mathscr{R}^n} g^*(\boldsymbol{r})\\
=\ &\max_{\rho_n\in[\sigma_n,1],\forall n\le N} \left\{ \max_{\boldsymbol{r}\in\mathscr{R}^N} g^*(\boldsymbol{r}) - \sum_{n=1}^{N-1} \rho_n [\max_{\boldsymbol{r}\in\mathscr{R}^{n+1}} g^*(\boldsymbol{r}) - \max_{\boldsymbol{r}\in\mathscr{R}^n} g^*(\boldsymbol{r})] \right\}\\
=\ &\max_{\boldsymbol{r}\in\mathscr{R}^N} g^*(\boldsymbol{r}) - \sum_{n=1}^{N-1} \sigma_n [\max_{\boldsymbol{r}\in\mathscr{R}^{n+1}} g^*(\boldsymbol{r}) - \max_{\boldsymbol{r}\in\mathscr{R}^n} g^*(\boldsymbol{r})]\\
=\ &\sum_{n=1}^{N} (\sigma_n - \sigma_{n-1}) \max_{\boldsymbol{r}^n\in\mathscr{R}^n, \boldsymbol{y}\in\mathcal{Y}(\boldsymbol{r}^n)} \min_{\boldsymbol{C}\in\mathcal{C}(\boldsymbol{r}^n,\boldsymbol{y})} g(\boldsymbol{r}^n, \boldsymbol{y}, \boldsymbol{C}).\end{aligned}$$

Proposition 5.1 is proved. □

A.15 Constraints (19c) and (19d)

Constraint (19c) represents a set of constraints related to the vector $\boldsymbol{y}^\phi$, and is composed of:

$$\begin{aligned}&\sum_{i'_\phi\in\mathcal{I}^\phi} y^\phi_{0i'_\phi} = 1, \forall \phi \in \Phi,\\
&\sum_{i'_\phi\in\mathcal{I}^\phi\setminus\{i_\phi\}} y^\phi_{i_\phi i'_\phi} - \sum_{i''_\phi\in\mathcal{I}\setminus\{i_\phi\}} y^\phi_{i''_\phi i_\phi} = 0, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi,\\
&\sum_{i_\phi\in\mathcal{I}^\phi} y^\phi_{i_\phi,|\mathcal{I}|+1} = 1, \forall \phi \in \Phi,\end{aligned}$$

$$\sum_{i_\phi \in \mathcal{I}^\phi \cup \{0\}} y^\phi_{i_\phi i'_\phi} = 1, \forall i'_\phi \in \mathcal{I}^\phi \backslash \{i_\phi\}, \forall \phi \in \Phi,$$
$$r_{i_\phi,\phi} + M(1 - y^\phi_{i_\phi i'_\phi}) \leq r_{i'_\phi,\phi}, \forall i_\phi \in \mathcal{I}^\phi, \forall i'_\phi \in \mathcal{I}^\phi \backslash \{i_\phi\}, \forall \phi \in \Phi,$$
$$y^\phi_{i_\phi i'_\phi} \in \{0,1\}, \forall i_\phi \in \mathcal{I}^\phi, \forall i'_\phi \in \mathcal{I}^\phi \backslash \{i_\phi\}, \forall \phi \in \Phi.$$

Constraint (19d) represents a set of constraints related to the vector $\boldsymbol{C}^\phi$, and is composed of:

$$C^\phi_{\max} = \sum_{i_\phi \in \mathcal{I}^\phi} C^\phi_{i_\phi} y^\phi_{i_\phi,|\mathcal{I}|+1}, \forall \phi \in \Phi, \tag{37}$$
$$C^\phi_{i'_\phi} = \max \left\{ r_{i'_\phi,\phi}, \sum_{i_\phi \in \mathcal{I}^\phi} C^\phi_{i_\phi} y^\phi_{i_\phi i'_\phi} \right\} + p^\phi_{i'_\phi}, \forall i'_\phi \in \mathcal{I}^\phi \backslash i_\phi, \forall \phi \in \Phi. \tag{38}$$

And the linearization of Constraints (37) and (38) is given by

$$C^\phi_{i_\phi} - M(1 - y_{i_\phi,|\mathcal{I}|+1}) \leq C^\phi_{\max}, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi,$$
$$C^\phi_{i_\phi} + M(1 - y_{i_\phi,|\mathcal{I}|+1}) \geq C^\phi_{\max}, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi,$$
$$C^\phi_{i_\phi} - M(1 - y^\phi_{i_\phi i'_\phi}) \leq v^\phi_{i'_\phi}, \forall i_\phi \in \mathcal{I}^\phi, \forall i'_\phi \in \mathcal{I}^\phi \backslash i_\phi, \forall \phi \in \Phi,$$
$$C^\phi_{i_\phi} + M(1 - y^\phi_{i_\phi i'_\phi}) \geq v^\phi_{i'_\phi}, \forall i_\phi \in \mathcal{I}^\phi, \forall i'_\phi \in \mathcal{I}^\phi \backslash i_\phi, \forall \phi \in \Phi,$$
$$r_{i_\phi,\phi} + p^\phi_{i_\phi} + M(1 - \delta^\phi_{i_\phi}) \geq C^\phi_{i_\phi}, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi,$$
$$v_{i_\phi,\phi} + p^\phi_{i_\phi} + M\delta^\phi_{i_\phi} \geq C^\phi_{i_\phi}, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi,$$
$$r_{i_\phi,\phi} - M\delta^\phi_{i_\phi} \leq v_{i_\phi,\phi}, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi,$$
$$r_{i_\phi,\phi} + p^\phi_{i_\phi} \leq C^\phi_{i_\phi}, v_{i_\phi,\phi} + p^\phi_{i_\phi} \leq C^\phi_{i_\phi}, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi,$$
$$\delta^\phi_{i_\phi} \in \{0,1\}, \forall i_\phi \in \mathcal{I}^\phi, \forall \phi \in \Phi.$$

A.16 Proof of Proposition 5.2

To prove Proposition 5.2, we establish an upper bound for $\max_{\boldsymbol{r} \in \mathscr{R}} g(\boldsymbol{r}, \boldsymbol{y}, \boldsymbol{C})$ that cannot be reached. We then demonstrate that the gap between this upper bound and $g(\tilde{\boldsymbol{r}}, \boldsymbol{y}, \boldsymbol{C})$ is no more than $\frac{\gamma}{|\mathcal{K}|} \lceil \frac{|\mathcal{I}|}{2} \rceil \lfloor \frac{|\mathcal{I}|}{2} \rfloor (\bar{p} - \underline{p})$.

Given that every vector within $\tilde{\boldsymbol{r}}$ represents the maximum corresponding value, we sort the vector in descending order and label the ith value as $r_{[i]}$. Consequently, the arrival time of the i-th task at the workstation is no later than $r_{[i]}$ in any element of the ambiguity set we define. As such, we can assign the arrival time of the ith task at the workstation to $r_{[i]}$,

which corresponds to the processing time derived from the processing times of all tasks, thereby maximizing the objective. By decomposing the processing time and release time of each task, we thus obtain an upper bound for the problem.

We then demonstrate that the maximum gap between this upper bound and the objective function derived from the feasible solution $\tilde{\boldsymbol{r}}$ is no greater than the maximum gap between this upper bound and the objective function derived from the feasible solution $\tilde{\boldsymbol{r}}$. This is shown using a proof method similar to that in Proposition 4.6. Specifically, when $\tilde{r}_1 = \tilde{r}_2 = \cdots = \tilde{r}_{|\mathcal{I}|}$, the tasks in the upper bound are processed according to the LPT (Longest Processing Time) rule. Meanwhile, the feasible solution $\tilde{\boldsymbol{r}}$, in which all tasks arrive simultaneously, can be processed in any order. The maximum gap is attained when the feasible solution takes the minimum value. Therefore, tasks in the feasible solution $\tilde{\boldsymbol{r}}$ are processed according to the SPT (Shortest Processing Time) rule. Hence, the maximum gap is proven in accordance with Proposition 4.6.

Finally, as $\tilde{\boldsymbol{r}} \in \mathscr{R}$, it is intuitive to have $g\left(\tilde{\boldsymbol{r}}, \boldsymbol{y}, \boldsymbol{C}\right) \leq \max_{\boldsymbol{r} \in \mathscr{R}} g\left(\boldsymbol{r}, \boldsymbol{y}, \boldsymbol{C}\right) = g\left(\boldsymbol{r}^*, \boldsymbol{y}, \boldsymbol{C}\right)$. Taken together, Proposition 5.2 is proved.

□

B Experiment Settings and Numerical Results

Each transportation task comprises three sub-tasks based on operation flow: unloaded travel, rack delivery, and rack return. The dataset contains 5,381 sub-task records, each including AMR ID, task ID, sub-task type, optimal travel distance, actual travel distance, actual travel time, route adjustments, optimal path, and actual path. The optimal path and distance represent the static A^* route and distance from the sub-task origin to the destination. Key descriptive statistics for these operational metrics across sub-task categories are summarized in Table 2, where the optimal number of turns is derived from the coordinates of the optimal path.

Table 2: Overview of Descriptive Data Statistics

Evaluator	Sub-Task Category	Average	Median	Minimum	Maximum
Number of Tasks	Pick-up	1897	-	-	-
	Delivery	1809	-	-	-
	Return	1683	-	-	-
Route Adjustments	Pick-up	1.45	1	0	8
	Delivery	1.80	2	0	5
	Return	2.20	2	0	7
Optimal Distance (m)	Pick-up	35.24	35.55	0	509.51
	Delivery	43.77	42.43	4.35	106.4
	Return	43.29	41.90	4.35	106.4
Optimal Number of Turns	Pick-up	2.29	2	0	30
	Delivery	4.96	5	1	19
	Return	5.25	5	1	22
Actual Transport Distance (m)	Pick-up	35.59	25.55	0.0	509.51
	Delivery	55.63	52.90	4.35	158.26
	Return	56.55	54.30	4.35	148.6
Actual Travel Time (s)	Pick-up	49.25	34.93	0	737.03
	Delivery	90.52	83.53	15.36	280.19
	Return	92.84	87.50	11.62	254.02

Table 3 provides a data sample for Robot 11701005 completing Task 11792. Sample

1 corresponds to the pick-up sub-task, Sample 2 to the delivery sub-task, and Sample 3 to the return sub-task. While optimal distance and path are predetermined, other metrics are derived from actual operations. Our objective is to leverage this data for designing the nested ambiguity sets for travel time in the distributionally robust counterpart. To explore data relationships, we generated the following statistical visualizations. Notably, since rack delivery and return both occur under loaded conditions, we combined them into a single "loaded state" category for analysis.

Table 3: Sample Data from Robotic Travel Dataset

Data Field	Sample 1	Sample 2	Sample 3
AMR ID	11701005	11701005	11701005
Task ID	11792	11792	11792
Sub-Task Type	Empty	Delivery	Pick-up
Optimal Travel Distance (m)	76.85	69.35	69.35
Actual Travel Distance (m)	76.85	74.75	77.05
Actual Travel Time (s)	80.619	109.974	131.657
Route Adjustments	0	1	2
Optimal Path Coordinates	[[217.07, 204.641], [223.82, 204.641], [223.82, 212.141], [280.07, 212.141], [280.07, 217.141], [281.42, 217.141]]	[[281.42, 217.141], [254.92, 217.141], [254.92, 183.391], [251.42, 183.391], [251.42, 179.141], [250.07, 179.141]]	[[250.07, 179.141], [250.07, 214.641], [280.07, 214.641], [280.07, 217.141], [281.42, 217.141]]
Actual Path Coordinates	[[217.07, 204.641], [223.82, 204.641], [223.82, 212.141], [280.07, 212.141], [280.07, 217.141], [281.42, 217.141]]	[[281.42, 217.141], [282.77, 217.141], [282.77, 190.891], [282.77, 190.891], [282.77, 183.391], [248.72, 183.391], [248.72, 182.141], [250.07, 182.141], [250.07, 179.141]]	[[250.07, 179.141], [250.07, 198.391], [250.07, 198.391], [250.07, 219.641], [254.92, 219.641], [254.92, 219.641], [282.77, 219.641], [282.77, 217.141], [281.42, 217.141]]

As optimal metrics are known in advance, we first analyzed their relationship with actual travel time after excluding extreme-distance tasks (final dataset: 5,243 sub-tasks). Using the number of turns derived from optimal paths, we generated Fig. 3 and Fig. 4 to examine how optimal distance and optimal number of turns influence actual travel time.

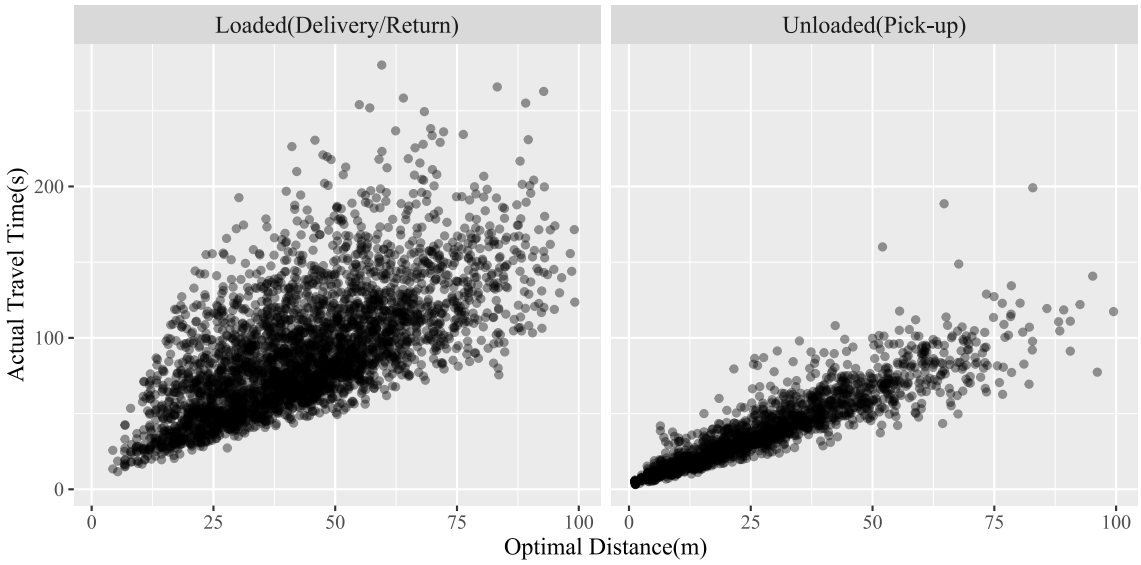


Figure 3: Scatter plot: Optimal path distance vs. actual travel time

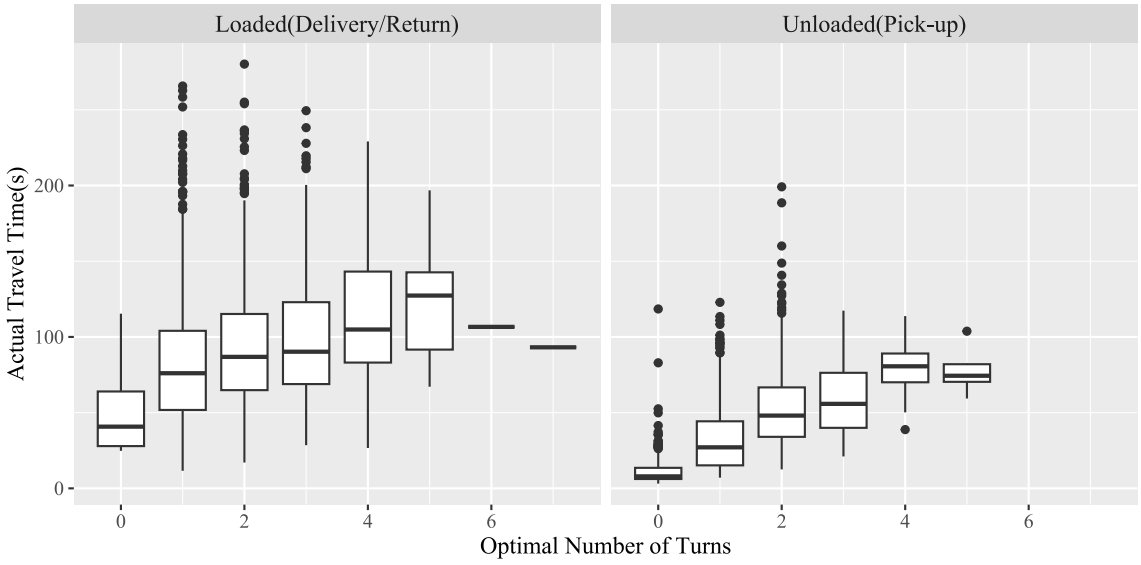


Figure 4: Box plot: Optimal number of turns vs. actual travel time

Fig. 3 reveals a strong positive correlation between travel time and optimal distance. The wider dispersion in the upper scatter region indicates prolonged times for some orders, suggesting that mean values may not capture this variability. Furthermore, loaded travel exhibits slower speeds and greater scatter compared to unloaded travel. Fig. 4 confirms that increased number of turns correspond to longer travel times, with loaded travel showing greater uncertainty. Fig. 5 displays the distribution of tasks by optimal number of turns.

Figure 5: Histogram: Task frequency by optimal number of turns

We subsequently analyzed how actual distance relates to travel time (Fig. 6). Despite exhibiting tighter clustering than optimal distance metrics, the metric is unavailable during task assignment. Thus, we focus on leveraging predetermined optimal metrics to enhance travel time characterization.

Given the complexity of precise time prediction and the right-skewed distribution of travel times (evidenced by elongated upper tails), we employed quantile regression to improve robustness. This approach generates parameters for different confidence levels, facilitating integration with the nested uncertainty sets construction. We first present single-variable quantile regression results relating travel time to optimal distance. Fig. 7 plots the cumulative distributions of travel time per meter for loaded and unloaded states. Both distributions are right-skewed, indicating occasional longer times. The 75th percentile times

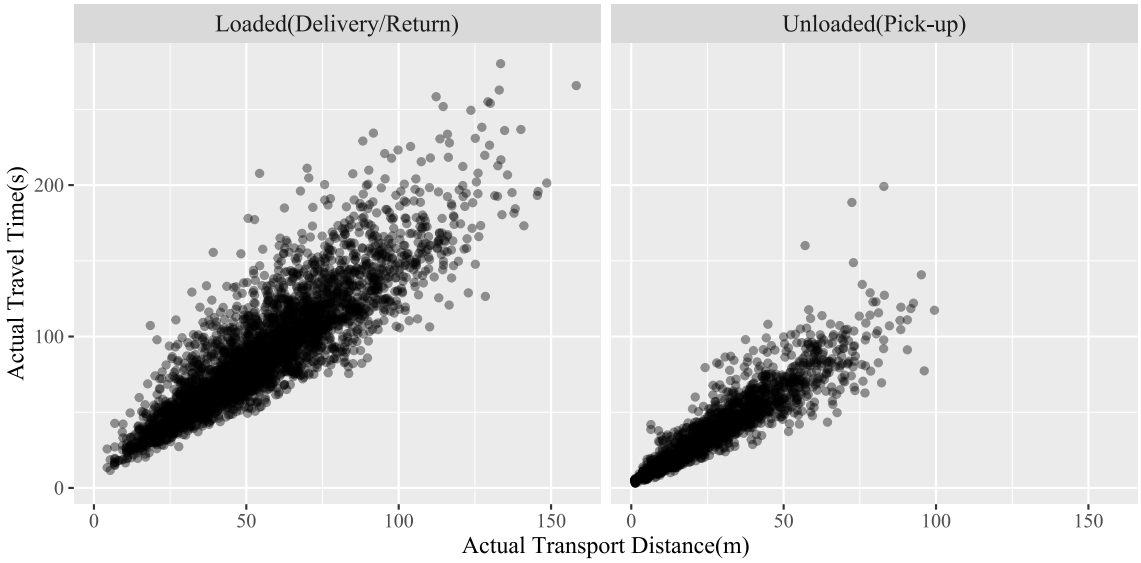


Figure 6: Scatter plot: Actual travel distance vs. actual travel time

per meter are 2.06s (unloaded) and 2.58s (loaded), represented by the blue reference lines.

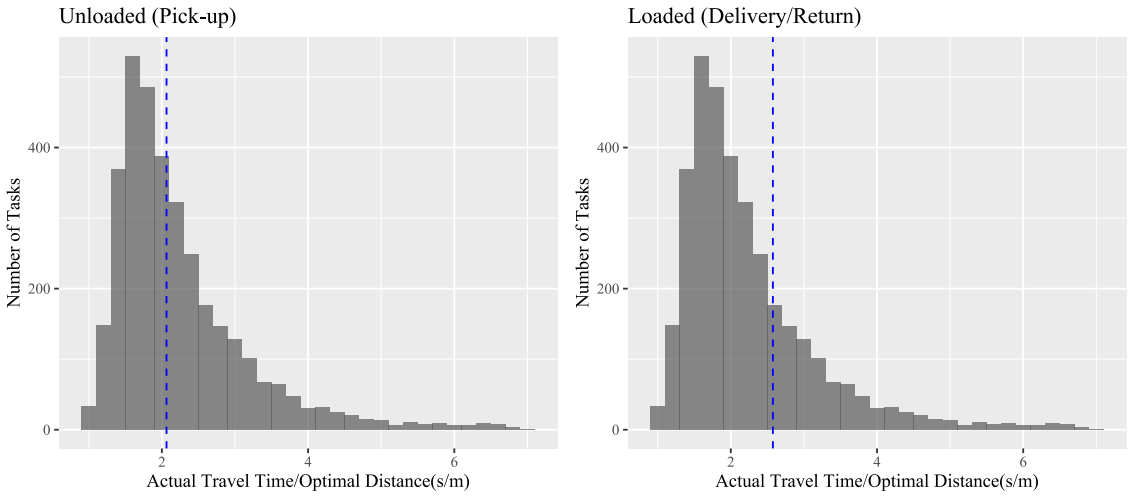


Figure 7: Cumulative distribution: Travel time per meter of optimal path

Fig. 8 shows quartile regression lines (blue) against linear regression (red), demonstrating quantile regression's superior encapsulation of data variability across diverse scenarios.

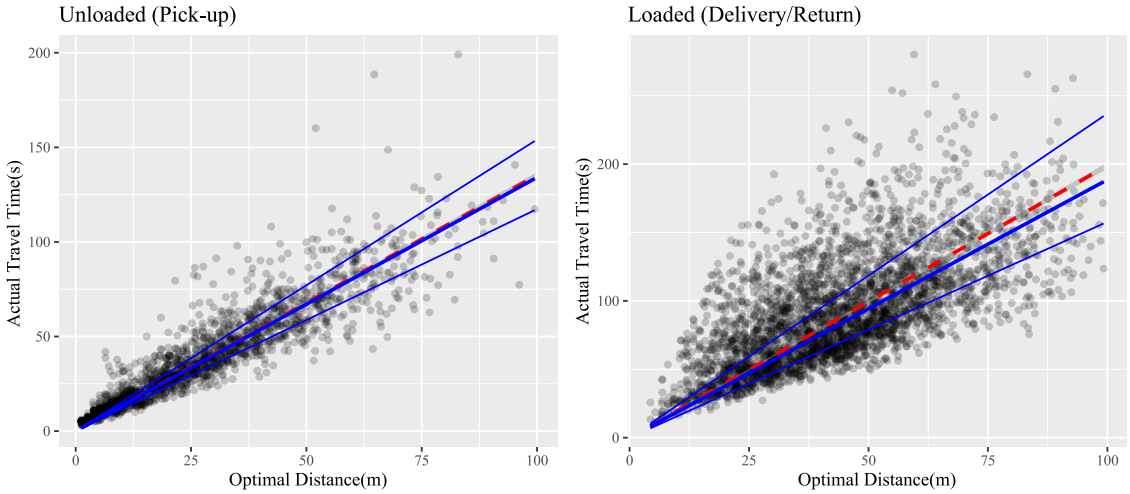


Figure 8: Quantile regression: Actual travel time vs. optimal distance

Table 4: Quantile Regression Coefficients by AMR Load Status

	Unloaded			**Loaded**			**Mixed**		
Variable	$q = 0.25$	$q = 0.50$	$q = 0.75$	$q = 0.25$	$q = 0.50$	$q = 0.75$	$q = 0.25$	$q = 0.50$	$q = 0.75$
Constant	1.8004	2.1096	2.8549	10.2291	15.2073	36.7511	-0.6716	6.0217	25.3485
Travel Distance (Unloaded)	1.0471	1.1852	1.3416	–	–	–	1.1620	1.2657	1.2585
Number of Turns (Unloaded)	1.6827	1.9927	2.2829	–	–	–	2.2586	1.3624	0.6493
Travel Distance (Loaded)	–	–	–	1.3220	1.4615	1.4752	1.5097	1.6511	1.7991
Number of Turns (Loaded)	–	–	–	0.7152	2.4934	2.5402	2.3879	2.7678	3.0656

Table 5: Experiment Settings on Workstations

Number of Workstations	Total Number of Tasks	Average Processing Time (s)
1	152	21.19
2	142	21.27
3	140	21.86
4	142	22.11
5	148	21.35
6	142	21.27
7	158	21.53
8	149	22.42
9	149	21.75

Table 6: Algorithm Comparison in Workstation Occupancy With Deterministic Travel Time

Number of Workstations	Greedy Algorithm	DRO Algorithm	Absolute Improvement	Relative Improvement
1	75.64%	83.26%	7.62%	10.07%
2	71.06%	82.35%	11.29%	15.89%
3	72.52%	79.72%	7.20%	9.93%
4	73.39%	81.24%	7.85%	10.70%
5	74.52%	84.54%	10.02%	13.45%
6	71.10%	77.89%	6.79%	9.55%
7	83.03%	88.36%	5.33%	6.42%
8	77.83%	86.73%	8.90%	11.44%
9	75.38%	85.37%	9.99%	13.25%
Average	74.94%	83.27%	8.33%	11.12%